

BÀI 8. LẬP TRÌNH VỚI GIAO THỨC MẠNG

8.1. Khái niệm Socket

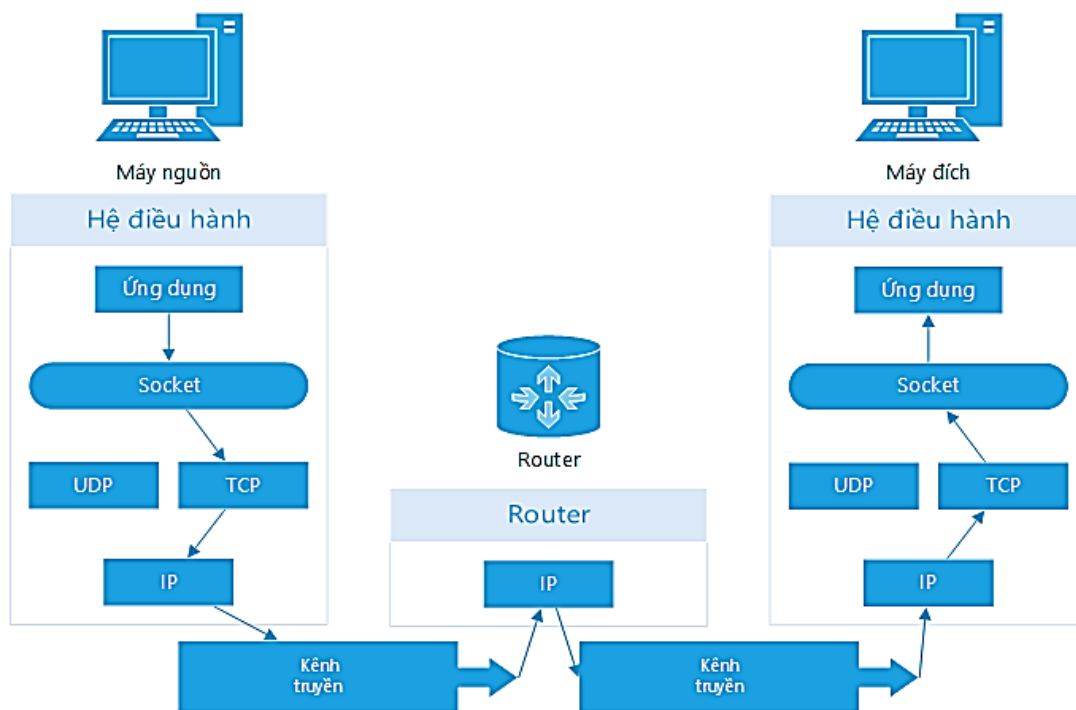
Lập trình với socket là nền tảng cho việc phát triển tất cả các loại phần mềm có sử dụng dịch vụ truyền thông mạng, cho phép chúng ta sử dụng các dịch vụ truyền thông mạng mà hệ thống cung cấp. Trong phần này chúng ta sẽ xem xét chi tiết khái niệm và vị trí của socket.

a. Góc nhìn truyền thông

Khi nhìn nhận từ khía cạnh truyền thông (đường đi của thông tin), socket có thể được hình dung như là một cánh cửa ngăn cách giữa chương trình ứng dụng (thuộc toàn quyền của người lập trình) và ngăn xếp giao thức mạng (thuộc quyền quản lý của hệ điều hành) (xem Hình 0.1).

Dữ liệu do chương trình tạo ra đi xuyên qua cánh cửa này để đến thế giới mạng, nơi nó sẽ được truyền tới nơi cần đến. Ở chiều ngược lại, dữ liệu từ mạng có thể đi xuyên qua cánh cửa này để tới được chương trình, nơi nó sẽ được xử lý.

Một khi dữ liệu đã đi xuyên qua cánh cửa này, người lập trình sẽ không kiểm soát được nữa mà hoàn toàn do các giao thức mạng nhào nặn để làm sao có thể truyền đi qua mạng một cách tốt nhất. Người lập trình chỉ có khả năng lựa chọn loại dịch vụ vận chuyển dữ liệu theo nhu cầu. Cụ thể hơn người lập trình chỉ có thể lựa chọn dịch vụ TCP hoặc dịch vụ UDP, đồng thời cung cấp tham số để các dịch vụ này phục vụ việc vận chuyển dữ liệu theo yêu cầu của mình.

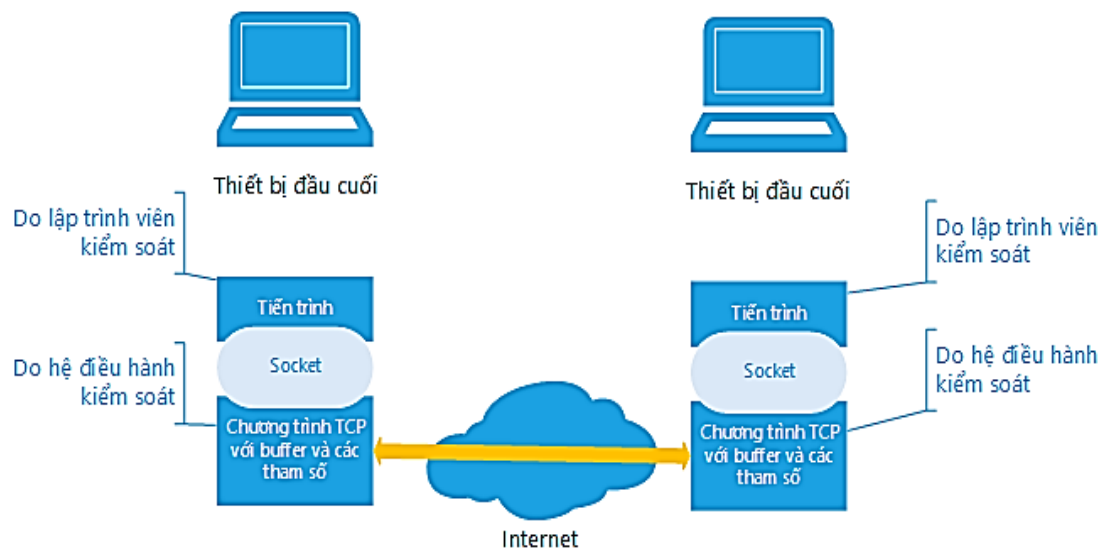


Hình 0.1. Đường đi của dữ liệu từ ứng dụng qua socket tới mạng

b. Góc nhìn mô hình mạng

Nhìn từ ứng dụng, socket là giao diện giữa ứng dụng và dịch vụ tầng giao vận trên mỗi máy. Trong mô hình mạng TCP/IP, socket có thể xem như giao diện giữa tầng ứng dụng và tầng chuyển vận (xem Hình 0.2).

Người phát triển ứng dụng có toàn quyền kiểm soát phía tầng ứng dụng của socket nhưng không thể kiểm soát phía tầng giao vận của socket. Ở phía tầng giao vận, người phát triển ứng dụng chỉ có thể lựa chọn giao thức của tầng này và điều chỉnh một vài tham số (kích thước tối đa của bộ nhớ đệm, kích thước tối đa của các segment dữ liệu).



Hình 0.2. Quan hệ tiến trình – socket – dịch vụ TCP trên thiết bị đầu cuối

Trong hai cách nhìn nhận trên, socket đóng vai trò điểm đầu và điểm cuối của quá trình truyền thông mạng, cũng như phân tách giữa tiến trình (chương trình ứng dụng) và dịch vụ vận chuyển của mạng.

So sánh một cách hình tượng, mỗi tiến trình có thể so sánh với một ngôi nhà, còn socket chính là cửa của ngôi nhà đó, và môi trường mạng truyền dữ liệu tương tự như một dịch vụ chuyển phát nhanh đặt ở ngay cửa của ngôi nhà.

c. Góc nhìn lập trình

Đối với người lập trình ứng dụng, socket có thể được hình dung là một giao diện lập trình ứng dụng (API) để gọi tới các chương trình con của hệ điều hành.

Trong các bài trước chúng ta đã nhắc tới giao thức TCP hay giao thức UDP. Về bản chất, TCP hay UDP đều là những chương trình được tích hợp sẵn trong các hệ điều hành hiện đại.

Tương tự như các chương trình hệ thống khác, TCP hay UDP (và cả IP) cũng cung cấp API để người lập trình có thể sử dụng được các chương trình này.

Khi học lập trình socket, chúng ta sẽ nhìn nhận khái niệm socket chủ yếu theo góc nhìn này. Việc học lập trình socket khi đó thực chất là việc học làm việc với các API của hệ thống để gọi các dịch vụ truyền thông.

Socket API đầu tiên được xây dựng bởi đại học Berkeley cho hệ điều hành BSD nên thường được gọi là BSD socket hay Berkeley socket. Sau đó Microsoft tham khảo và tạo ra các socket API dành cho hệ điều hành windows, gọi là windows socket, hay thường gọi tắt là winsock. Winsock là bộ API tiêu chuẩn để lập trình mạng trong Windows.

Cũng vì lý do này mà các hàm socket API trên các hệ điều hành hiện nay đều có tên gọi gần giống nhau. Nắm bắt được cách lập trình với socket API trên một nền tảng này có thể dễ dàng tiếp cận với các nền tảng khác.

d. Phân loại socket

Do có 2 loại dịch vụ hỗ trợ trao đổi dữ liệu qua mạng như đã nói, socket cũng được chia làm 2 loại chủ yếu:

Socket để sử dụng dịch vụ TCP, gọi là TCP socket:

Bởi vì TCP cung cấp dịch vụ truyền dữ liệu theo một liên kết ảo giữa hai tiến trình, TCP socket còn được gọi là socket hướng kết nối (connection-oriented socket). Do dữ liệu truyền theo liên kết TCP được hình dung như một chuỗi byte liên tục, loại socket này còn có một tên gọi khác là socket hướng dòng (stream socket).

Socket để sử dụng dịch vụ UDP, gọi là UDP socket:

Do UDP không tạo liên kết mà truyền dữ liệu theo các gói (datagram) độc lập, UDP socket được còn được gọi là socket phi liên kết (connectionless socket) hay dgram socket.

Đối với người phát triển ứng dụng hệ thống còn một loại socket nữa, gọi là socket thô (raw socket). Loại socket này cho phép gọi thẳng đến chương trình IP mà bỏ qua TCP hoặc UDP.

Bình thường, dữ liệu sau khi đến chương trình IP sẽ được chuyển tiếp lên chương trình TCP hay UDP, sau đó mới được chuyển tới ứng dụng. Khi sử dụng socket thô, ta có thể bỏ qua TCP/UDP mà trực tiếp nhận gói tin “thô” từ IP. Loại socket này giúp chúng ta phát triển một số ứng dụng đặc biệt như ping, trace route.

e. Winsock

Các hệ điều hành đầu tiên của Microsoft (như MS-DOS và các phiên bản đầu của Microsoft Windows) có cung cấp khả năng kết nối mạng nhưng rất hạn chế, chủ yếu dựa trên NetBIOS. Trên thực tế, tại thời điểm đó, Microsoft không cung cấp hỗ trợ cho giao thức TCP / IP.

Một số nhóm và nhà cung cấp thương mại (như nhóm PC / IP tại MIT, FTP Software, Sun Microsystems, Ungermann-Bass và Excelan) đã giới thiệu các sản phẩm TCP / IP cho MS-DOS, ở dạng một bộ phận của gói phần cứng / phần mềm. Khi Windows 2.0 được phát hành, một số nhóm khác (như Distinct và NetManage) cũng tham gia cung cấp TCP / IP cho Windows.

Một vấn đề rất lớn gặp phải là mỗi nhà cung cấp lại sử dụng bộ API của riêng mình mà không có một mô hình lập trình tiêu chuẩn thống nhất. Đến năm 1991, Martin Hall của JSB Software (sau này đổi tên thành Stardust Technologies) đề xuất Windows Sockets API và trở thành tiêu chuẩn cho lập trình ứng dụng mạng trong Windows.

Windows Sockets API (WSA), sau rút gọn thành Winsock, là một đặc tả kỹ thuật xác định cách phần mềm mạng trong Windows truy cập các dịch vụ mạng, đặc biệt là TCP / IP. Winsock định nghĩa giao diện chuẩn giữa ứng dụng (ví dụ chương trình FTP client hoặc trình duyệt web) và giao thức

TCP / IP bên dưới. Winsock API được đặt trong file winsock.dll (16 bit) hoặc wsock32.dll (32 bit) trong thư mục hệ thống.

Windows Sockets định nghĩa hai giao diện:

- API cho các nhà phát triển phần mềm ứng dụng, và SPI cho các nhà phát triển phần mềm hệ thống (để thêm các giao thức mới vào hệ thống). API đảm bảo cho ứng dụng có thể hoạt động với cài đặt của giao thức (protocol implementation) từ bất kỳ nhà cung cấp phần mềm mạng nào.
- SPI đảm bảo cho một cài đặt của giao thức phù hợp có thể được thêm vào hệ thống Windows, và nếu một ứng dụng tuân thủ API thì có thể sử dụng được giao thức mới. Tuy nhiên, hiện nay SPI rất hiếm được sử dụng do trong tất cả các phiên bản Windows gần đây, Microsoft đã hỗ trợ đầy đủ ngăn xếp giao thức TCP / IP và không có nhiều người quan tâm xây dựng các giao thức khác với TCP / IP.

Code và thiết kế của Windows Socket dựa trên tham khảo BSD Socket và cung cấp chức năng bổ sung để API tuân thủ theo mô hình lập trình Windows. Windows Sockets API bao gồm gần như tất cả các tính năng của BSD Socket API. Do đó, khi nắm được kỹ thuật lập trình với một bộ socket API có thể dễ dàng tiếp cận với các bộ socket API khác.

Ngoài ra, nhiều công cụ phát triển phần mềm ứng dụng hiện đại cũng tạo ra các lớp “vỏ bọc” (wrapper) riêng xung quanh bộ socket API để đơn giản hóa và nâng cao hiệu quả trong phát triển các phần mềm ứng dụng mạng.

f. Hạn chế của lập trình socket với winsock API

Như trên vừa trình bày, winsock là bộ API tiêu chuẩn để lập trình mạng trong windows. Tuy nhiên, việc lập trình ứng dụng với winsock có những khó khăn nhất định.

Thứ nhất, các API hệ thống thường rất phức tạp với rất nhiều tham số gây khó khăn cho việc lập trình. Để đảm bảo tính linh hoạt, mỗi API đều chứa rất nhiều tham số, sử dụng nhiều kiểu dữ liệu hỗ trợ, cũng như có rất nhiều loại “magic constant”. Lập trình với socket API cũng không ngoại lệ.

Thứ hai, việc gọi đến các API của hệ thống thường chỉ phù hợp khi lập trình với một số ngôn ngữ và công nghệ nhất định. Ví dụ khi sử dụng C/C++/Delphi xây dựng ứng dụng native cho windows sẽ dễ dàng truy cập các API này hơn. Tuy nhiên, sử dụng các ngôn ngữ và công cụ bậc “không cao” như C/C++ làm tăng thời gian phát triển ứng dụng (giảm năng suất).

Các công nghệ phát triển ứng dụng hiện đại thường hạn chế việc truy xuất trực tiếp đến các API của hệ thống. Thay vào đó, các công nghệ này thường tạo ra các “vỏ bọc” (wrapper) để giúp người lập trình gọi đến các API của hệ thống một cách dễ dàng hơn.

Khi sử dụng socket, bạn đang làm việc với dịch vụ TCP hoặc UDP của hệ thống để gửi và nhận dữ liệu. Để đạt được hiệu quả, bạn bắt buộc phải xây dựng một giao thức riêng của mình (được tính là giao thức thuộc tầng ứng dụng). Tạo ra các giao thức tầng ứng dụng không phải là một công việc đơn giản, đặc biệt nếu ứng dụng của bạn có những yêu cầu cao về mặt truyền thông (ví dụ, đối với game online).

Khi lập trình socket, bạn được (và phải) kiểm soát hoàn toàn quy trình truyền thông và dữ liệu. Đây là một con dao hai lưỡi với người lập trình, nhất là khi bạn không có nhiều kiến thức về mạng máy tính và truyền thông.

Lập trình socket được sử dụng nếu ứng dụng của bạn có những yêu cầu đặc biệt về truyền thông mà không có các giao thức ứng dụng nào phù hợp. Lập trình socket là bắt buộc nếu bạn muốn sáng tạo một giao thức tầng ứng dụng. Như đã nói, khi thực thi một giao thức ứng dụng, bạn bắt buộc phải sử dụng một trong hai dịch vụ giao vận. Việc lựa chọn dịch vụ nào phụ thuộc vào yêu cầu cụ thể của giao thức.

Như vậy, có thể nói rằng, mặc dù socket là nền tảng về truyền thông trong lập trình ứng dụng mạng, việc sử dụng trực tiếp socket là tương đối hạn chế. Hiện nay có rất nhiều giao thức truyền thông đa năng có thể sử dụng trong phát triển ứng dụng mạng. Các giao thức này được thực thi trên nhiều nền tảng và ngôn ngữ để tích hợp vào nhiều loại ứng dụng khác nhau. Vì vậy, việc sử dụng trực tiếp socket trong phát triển ứng dụng là không phổ biến.

8.2. Lập trình với socket UDP

a. Thực hành

Bài toán: Viết một ứng dụng dòng lệnh đơn giản với *giao diện dòng lệnh* (console) theo mô hình chủ / khách sử dụng mô hình truy vấn / phản hồi đáp ứng các yêu cầu sau:

- Client cho phép người dùng nhập một chuỗi ký tự từ bàn phím và gửi chuỗi ký tự cho server;
- Server nhận chuỗi ký tự, chuyển đổi tất cả ký tự thành dạng in hoa và gửi chuỗi kết quả lại cho client;
- Client nhận kết quả và hiển thị lại cho người dùng.

Cùng thực hiện theo các bước sau đây.

Bước 1. Tạo hai dự án kiểu Console App, một dự án đặt tên là Client, dự án kia đặt tên là Server. Hai dự án đặt trong giải pháp UdpSocket

Bước 2. Thiết lập để đồng thời chạy debug cả hai chương trình.

Bước 3. Mở file Program.cs trong dự án Client và viết code như sau. Chú ý đọc kỹ các comment để nắm được ý nghĩa của từng lệnh.

Client \ Program.cs

```
using System;
using System.Net; // để sử dụng lớp IPAddress, IPEndPoint
using System.Net.Sockets; // để sử dụng lớp Socket
using System.Text; // để sử dụng lớp Encoding
namespace Client;
internal class Program {
    private static void Main(string[] args) {
        Console.Title = "UDP Client";
        // yêu cầu người dùng nhập ip của server
        Console.Write("Server IP address: ");
        var serverIpStr = Console.ReadLine();
        // chuyển đổi chuỗi ký tự thành object thuộc kiểu IPAddress
```

```

var serverIp = IPAddress.Parse(serverIpStr);
// yêu cầu người dùng nhập cổng của server
Console.WriteLine("Server port: ");
var serverPortStr = Console.ReadLine();
// chuyển chuỗi ký tự thành biến kiểu int
var serverPort = int.Parse(serverPortStr);
// đây là "địa chỉ" của tiến trình server trên mạng
// mỗi endpoint chứa ip của host và port của tiến trình
var serverEndpoint = new IPEndPoint(serverIp, serverPort);
var size = 1024; // kích thước của bộ đệm
var receiveBuffer = new byte[size]; // mảng byte làm bộ đệm
while (true) {
    // yêu cầu người dùng nhập một chuỗi
    Console.ForegroundColor = ConsoleColor.Green;
    Console.Write("# Text >>> ");
    Console.ResetColor();
    var text = Console.ReadLine();
    // khởi tạo object của lớp socket để sử dụng dịch vụ UDP
    // lưu ý SocketType của UDP là Dgram (datagram)
    var socket = new Socket(SocketType.Dgram, ProtocolType.UDP);
    // biến đổi chuỗi thành mảng byte
    var sendBuffer = Encoding.ASCII.GetBytes(text);
    // gửi mảng byte trên đến tiến trình server
    socket.SendTo(sendBuffer, serverEndpoint);
    // endpoint này chỉ dùng khi nhận dữ liệu
    IPEndPoint dummyEndpoint = new IPEndPoint(IPAddress.Any, 0);
    // nhận mảng byte từ dịch vụ UDP và lưu vào bộ đệm
    // biến dummyEndpoint lưu lại địa chỉ của tiến trình nguồn
    // tuy nhiên, ở đây chúng ta đã biết tiến trình nguồn là Server
    // do đó dummyEndpoint không có giá trị sử dụng
    var length = socket.ReceiveFrom(receiveBuffer, ref dummyEndpoint);
    // chuyển đổi mảng byte về chuỗi
    var result = Encoding.ASCII.GetString(receiveBuffer, 0, length);
    // xóa bộ đệm (để lần sau sử dụng cho yên tâm)
    Array.Clear(receiveBuffer, 0, size);
    // đóng socket và giải phóng tài nguyên
    socket.Close();
    // in kết quả ra màn hình
    Console.WriteLine($">>> {result}");
}
}
}

```

Bước 4. Mở file Program.cs trong dự án Server và viết code như sau:

Server \ Program.cs

```

using System;
using System.Net;
using System.Net.Sockets;
using System.Text;
namespace Server;
internal class Program {
    private static void Main(string[] args) {
        Console.Title = "UDP Server";
        // giá trị Any của IPAddress tương ứng với IP của tất cả các giao diện mạng trên
        máy
        var localIp = IPAddress.Any;
        // tiến trình server sẽ sử dụng cổng 1308
        var localPort = 1308;
        // biến này sẽ chứa "địa chỉ" của tiến trình server trên mạng
    }
}

```

```

var localEndPoint = new IPEndPoint(localIp, localPort);
// yêu cầu hệ điều hành cho phép chiếm dụng cổng 1308
// server sẽ nghe trên tất cả các mạng mà máy tính này kết nối tới
// chỉ cần gói tin udp đến cổng 1308, tiến trình server sẽ nhận được
// một overload khác của hàm tạo Socket
// InterNetwork là họ địa chỉ dành cho IPv4
var socket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram,
ProtocolType.UDP);
socket.Bind(localEndPoint);
Console.WriteLine($"Local socket bind to {localEndPoint}. Waiting for request
...");

var size = 1024;
var receiveBuffer = new byte[size];
while (true) {
    // biến này sẽ chứa địa chỉ của tiến trình client nào gửi gói tin tới
   EndPoint remoteEndPoint = new IPEndPoint(IPAddress.Any, 0);
    // khi nhận được gói tin nào sẽ lưu lại địa chỉ của tiến trình client
    var length = socket.ReceiveFrom(receiveBuffer, ref remoteEndPoint);
    var text = Encoding.ASCII.GetString(receiveBuffer, 0, length);
    Console.WriteLine($"Received from {remoteEndPoint}: {text}");
    // chuyển chuỗi thành dạng in hoa
    var result = text.ToUpper();
    var sendBuffer = Encoding.ASCII.GetBytes(result);
    // gửi kết quả lại cho client
    socket.SendTo(sendBuffer, remoteEndPoint);
    Array.Clear(receiveBuffer, 0, size);
}
}
}

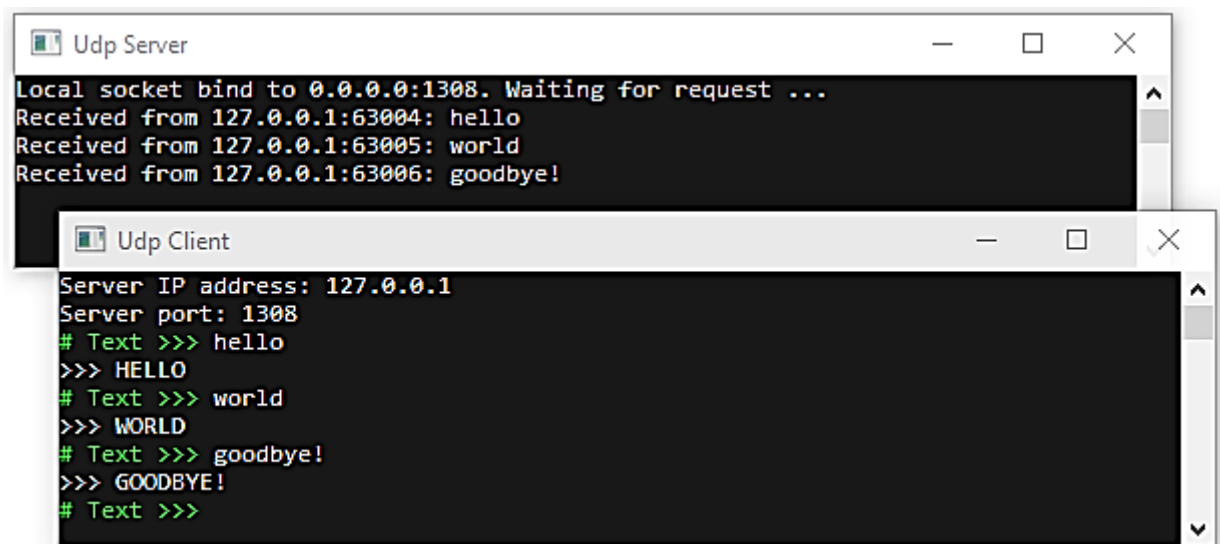
```

Bước 5. Chạy thử ứng dụng bằng phím F5 hoặc tổ hợp Ctrl+F5. Khi này, chương trình server sẽ chạy trước, chương trình client sẽ chạy ngay sau server (Hình 0.1).

Server sẽ viết ra thông báo về IP và giá trị cổng mà nó đang chờ request. Client sẽ yêu cầu người dùng nhập địa chỉ IP và số cổng của server. Vì server đang nghe tất cả các giao diện mạng và client đang chạy trên cùng máy vật lý với server, client có thể sử dụng địa chỉ loopback (127.0.0.1) và số cổng 1308.

Nhập bất kỳ chuỗi ký tự nào vào dấu nhắc lệnh của client. Ngay sau đó sẽ nhận được kết quả là chuỗi ký tự đã chuyển thành chữ hoa.

Lưu ý, khi Server hoạt động lần đầu tiên, hệ điều hành sẽ yêu cầu người dùng cấp phép cho Server chiếm dụng cổng 1308. Nếu chạy Server từ một tài khoản bị hạn chế quyền sẽ không thực hiện được yêu cầu này, và Server sẽ bị đóng lại ngay sau đó. Từ lần hoạt động sau sẽ không cần trả lời cấp phép nữa.



The image shows two overlapping terminal windows. The top window, titled 'Udp Server', displays the following text: 'Local socket bind to 0.0.0.0:1308. Waiting for request ...', 'Received from 127.0.0.1:63004: hello', 'Received from 127.0.0.1:63005: world', and 'Received from 127.0.0.1:63006: goodbye!'. The bottom window, titled 'Udp Client', displays: 'Server IP address: 127.0.0.1', 'Server port: 1308', '# Text >>> hello', '>>> HELLO', '# Text >>> world', '>>> WORLD', '# Text >>> goodbye!', '>>> GOODBYE!', and '# Text >>>'. The client window has a green prompt character '#' before each input line.

Hình 0.1. Ảnh chụp màn hình khi ứng dụng hoạt động

Để ý rằng mỗi lần Client phát đi một chuỗi ký tự, giá trị cổng của nó lại thay đổi. Trong ví dụ trên, client lần lượt sử dụng các cổng UDP là 63004, 63005, 63006 cho ba lần truy vấn.

b. Quy trình xây dựng ứng dụng khai thác UDP socket

Trong ví dụ trên chúng ta đã vận dụng kỹ thuật lập trình socket UDP để xây dựng ra một ứng dụng mạng theo mô hình chủ / khách đơn giản đầu tiên. Logic của cả hai ứng dụng được trình bày trong Hình 0.2.

Tiến trình chủ thực hiện các thao tác:

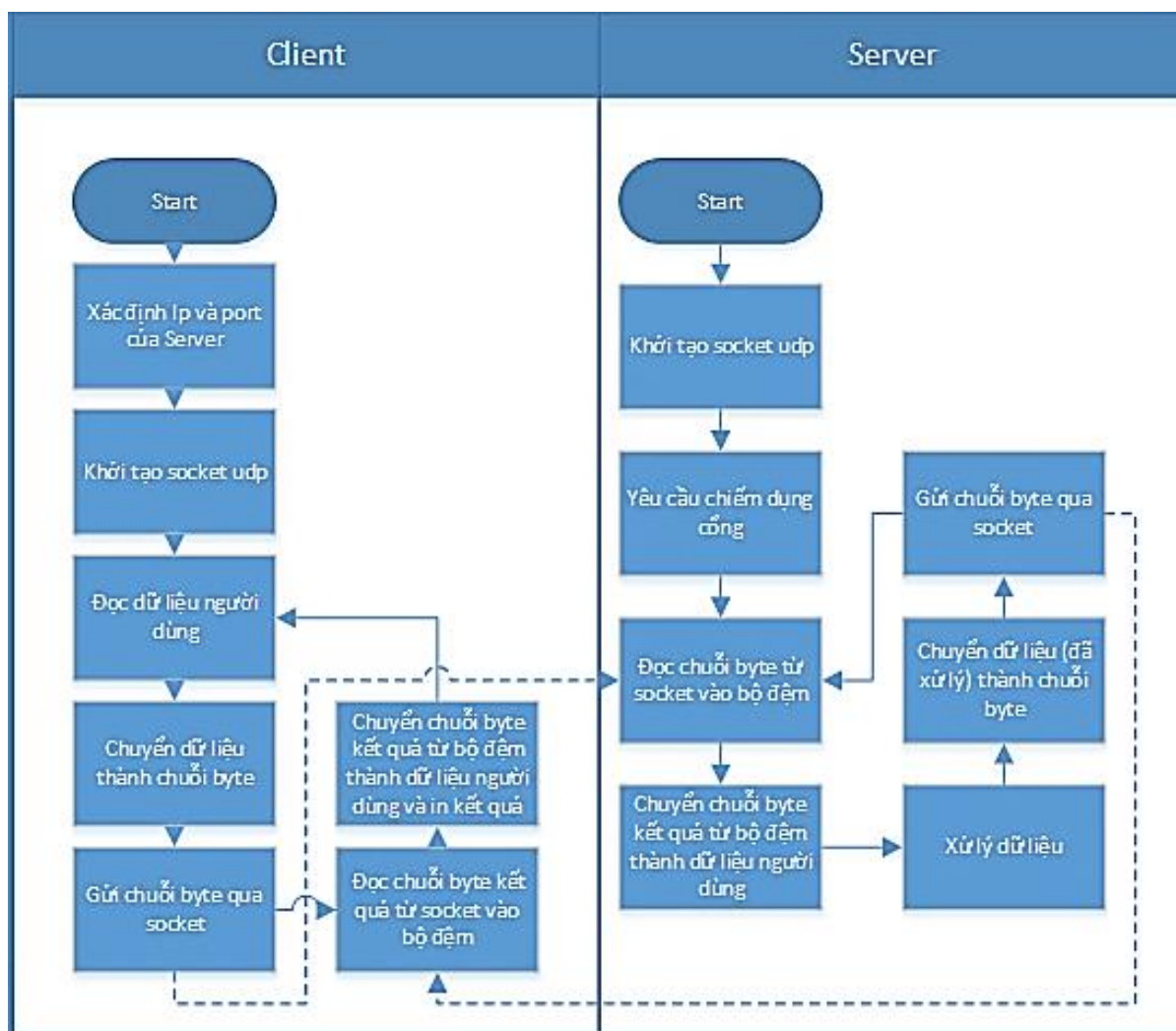
- 1) Khởi tạo socket UDP
- 2) Yêu cầu hệ điều hành cho phép chiếm dụng 1 cổng dịch vụ UDP cố định
- 3) Chờ đọc dữ liệu từ socket UDP
- 4) Nếu đọc được dữ liệu sẽ chuyển dữ liệu (ở dạng chuỗi byte) về dạng dữ liệu cấp cao của ngôn ngữ lập trình
- 5) Tiến hành xử lý dữ liệu
- 6) Sinh ra phản hồi (ở dạng ở liệu cấp cao của ngôn ngữ)
- 7) Chuyển đổi dữ liệu phản hồi về dạng chuỗi byte
- 8) Gửi dữ liệu trở lại cho client qua socket UDP

Các bước từ 3 đến 8 sẽ được lặp lại vô tận để đảm bảo server luôn luôn sẵn sàng phục vụ client.

Tiến trình khách thực hiện các thao tác:

- 1) Yêu cầu cung cấp địa chỉ IP và số cổng Port của server
- 2) Khởi tạo socket UDP
- 3) Yêu cầu người dùng cung cấp dữ liệu cần chuyển đi
- 4) Chuyển dữ liệu người dùng (là dạng dữ liệu cấp cao của ngôn ngữ lập trình) thành mảng byte
- 5) Gửi chuỗi byte qua UDP socket

- 6) Chờ đọc phản hồi của server
- 7) Nếu đọc xong dữ liệu, chuyển đổi chuỗi byte phản hồi thành dữ liệu cao cấp
- 8) Xử lý kết quả (vd, in ra màn hình)



Hình 0.2. Sơ đồ hoạt động của phần mềm

Ở đây chúng ta có thể thấy một sự khác biệt lớn nhất giữa client và server. Server phải yêu cầu chiếm dụng cổng cố định sau khi khởi tạo socket, còn client thì không cần. Việc gửi và nhận dữ liệu trên client và server là giống nhau.

Ở đây chúng ta cũng vận dụng mô hình trao đổi thông điệp kiểu truy vấn / phản hồi. Client là bên phát truy vấn tới server; Server phản hồi trở lại cho client với mỗi truy vấn. Truy vấn và phản hồi phải đi theo mô hình đồng bộ theo thứ tự truy vấn -> phản hồi.

Theo mô hình này chúng ta phải tạo ra hai cấu trúc dữ liệu riêng cho truy vấn và phản hồi. Tuy nhiên, trong bài toán này, cấu trúc dữ liệu thực tế chỉ là các chuỗi ký tự đơn giản.

c. Kỹ thuật lập trình UDP socket cơ bản

Tiếp theo đây chúng ta sẽ phân tích chi tiết kỹ thuật lập trình UDP socket cơ bản trong .NET, bao gồm cách khởi tạo, cách gửi nhận dữ liệu, cách đóng socket. Tất cả phân tích này thực hiện đối với code của phần thực hành đã thực hiện ở bài trước.

Trong phần thực hành ở bài trước chúng ta đã xây dựng được một ứng dụng mạng đầu tiên theo mô hình client/server sử dụng socket UDP. Chúng ta sẽ phân tích chi tiết code ứng dụng này để hiểu rõ hơn kỹ thuật lập trình với socket UDP trong C#.

Khởi tạo socket UDP

Đối với cả client và server trước khi có thể làm việc với dịch vụ UDP đều phải thực hiện khởi tạo socket UDP.

Có hai overload để khởi tạo đối tượng của lớp Socket. Trong phần thực hành trên chúng ta đã sử dụng cả hai overload này cho client và server:

```
var socket = new Socket(SocketType.Dgram, ProtocolType.UDP);  
var socket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.UDP);
```

Hai overload này đều yêu cầu cung cấp thông tin về loại socket (SocketType) và loại giao thức (ProtocolType).

SocketType là một kiểu liệt kê (enum) xác định loại socket sẽ được sử dụng. SocketType có thể nhận một trong các giá trị sau: Dgram (dành cho socket UDP), Raw (dành cho socket thô), Stream (dành cho socket TCP).

ProtocolType là một kiểu liệt kê thông báo cho Windows Socket API về loại giao thức sẽ được sử dụng trong socket và phải được lựa chọn phù hợp với SocketType. ProtocolType có rất nhiều giá trị khác nhau: TCP (giao thức TCP), UDP (giao thức UDP), IPv4 (giao thức IPv4), IPv6 (giao thức IPv6), Raw (gói tin IP thô), v.v..

Đối với socket UDP, giá trị SocketType phải là Dgram, ProtocolType là UDP.

Overload thứ hai yêu cầu cung cấp thêm thông tin về họ địa chỉ (AddressFamily) được sử dụng. AddressFamily là một kiểu liệt kê xác định loại phương pháp đánh địa chỉ mạng nào được sử dụng cho socket, trong đó InterNetwork là giá trị tương ứng với loại địa chỉ IPv4, InternetWorkV6 là giá trị tương ứng với loại địa chỉ IPv6.

Chiếm dụng cổng cho server

Một đối tượng của lớp Socket sau khi được khởi tạo với tham số phù hợp với giao thức UDP đã sẵn sàng để sử dụng dịch vụ vận chuyển (đầu cuối – đầu cuối, tức là tiến trình – tiến trình) của UDP.

Việc sử dụng này có sự khác biệt ở Client và Server:

- Client có thể ngay lập tức sử dụng socket để gửi gói tin tới Server.
- Server phải yêu cầu hệ điều hành cho phép chiếm dụng một cổng để chờ nghe gói tin đến từ Client.

Client và Server phải thỏa thuận với nhau về giá trị cổng này (tức là Client phải biết được Server đang chiếm dụng và chờ nghe ở cổng nào).

Việc yêu cầu chiếm dụng cổng này được thực hiện bởi chuỗi lệnh sau trong code Server:

```
var localIp = IPAddress.Any;  
var localPort = 1308;  
var localEndPoint = new IPEndPoint(localIp, localPort);  
var socket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.UDP);  
socket.Bind(localEndPoint);
```

Khi viết code yêu cầu chiếm dụng cổng dịch vụ UDP chúng ta gặp hai class mới: **IPAddress** và **IPEndPoint**.

Lớp **IPAddress** là một lớp hỗ trợ để làm việc với địa chỉ IPv4 như chuyển đổi qua lại giữa các cách thức biểu diễn địa chỉ IP (chuỗi ký tự, danh sách các octet của IP). Lớp **IPAddress** chứa danh sách các hằng số IP đặc biệt (Loopback và Broadcast cho IPv4, IPv6Loopback, IPv6Broadcast). Hai giá trị đặc biệt của **IPAddress** là **Any** (đại diện cho IP của tất cả các giao diện mạng) và **None** (không liên kết với bất kỳ giao diện mạng nào).

Lớp **IPEndPoint** hỗ trợ lưu trữ thông tin về điểm cuối dùng khi khởi tạo socket, thực hiện kết nối TCP, hoặc để thực hiện truyền thông qua UDP. Do socket là điểm đầu và điểm cuối của truyền thông liên tiến trình end-to-end, mỗi điểm cuối chứa địa chỉ của tiến trình trên liên mạng, bao gồm địa chỉ IP của máy và một cổng đại diện cho tiến trình.

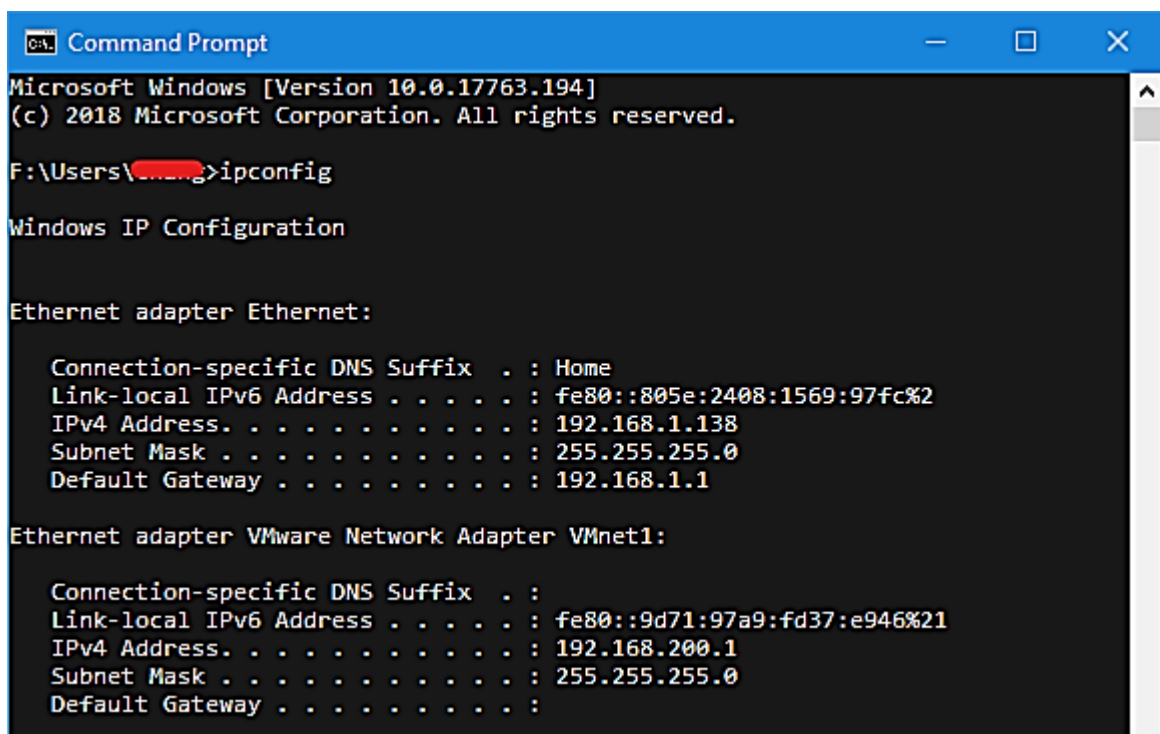
Ở Server phải dùng lệnh **Bind** của **Socket** để yêu cầu hệ điều hành cho phép chiếm dụng một cổng. Lệnh **Bind** yêu cầu cung cấp một đối tượng thuộc kiểu **IPEndPoint** làm tham số. Như trên đã nói, một endpoint chứa địa chỉ IP của host và số cổng mà tiến trình cần sử dụng.

Nếu cổng đang rồi (chưa có tiến trình nào sử dụng) và chương trình được cấp quyền thì nó sẽ chiếm dụng cổng này. Nếu cổng đã bị chiếm dụng bởi một tiến trình khác sẽ xảy ra lỗi. Nếu người dùng không có đủ quyền khi chạy ứng dụng thì chương trình Server cũng không chạy được tiếp.

Địa chỉ IP có thể là địa chỉ đang được gán cho bất kỳ NIC (Network Interface Card, card mạng) nào. Để xem được địa chỉ IP đang gán cho các NIC có thể dùng chương trình **IpConfig** của hệ thống từ **Command Prompt**.

Nếu một máy tính có nhiều card mạng, mỗi card được nối với một mạng và sẽ có IP riêng của mạng đó. Khi lựa chọn địa chỉ IP cụ thể nào, Server sẽ chỉ nhận gói tin đến từ mạng tương ứng.

Để Server có thể nhận gói tin đến từ tất cả các mạng, lớp **IPAddress** cung cấp giá trị **Any** (**IPAddress.Any** mà chúng ta đã sử dụng ở phần thực hành trên) dùng cho các mạng IPv4 hoặc **IPv6Any** dùng cho mạng IPv6.



```
Microsoft Windows [Version 10.0.17763.194]
(c) 2018 Microsoft Corporation. All rights reserved.

F:\Users\...>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : Home
    Link-local IPv6 Address . . . . . : fe80::805e:2408:1569:97fc%2
    IPv4 Address. . . . . : 192.168.1.138
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.1.1

Ethernet adapter VMware Network Adapter VMnet1:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::9d71:97a9:fd37:e946%21
    IPv4 Address. . . . . : 192.168.200.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :
```

Hình 0.3. Xem thông tin địa chỉ IP với lệnh IPConfig

Như vậy, trong đoạn code của Server ở trên, chúng ta yêu cầu hệ điều hành cho phép sử dụng cổng 1308 để lắng nghe gói tin đến từ tất cả các mạng mà máy tính kết nối tới.

Gửi dữ liệu qua socket UDP

Để gửi dữ liệu qua socket UDP chúng ta phải thực hiện hai thao tác:

- chuyển đổi dữ liệu người dùng thành chuỗi byte;
- phát lệnh gửi dữ liệu bằng phương thức SendTo của lớp Socket.

Trong phần thực hành, code gửi dữ liệu ở Client như sau:

```
// biến đổi chuỗi thành mảng byte
var sendBuffer = Encoding.ASCII.GetBytes(text);
// gửi mảng byte trên đến tiến trình server
socket.SendTo(sendBuffer, serverEndpoint);
```

và code ở Server:

```
// chuyển chuỗi thành dạng in hoa
var result = text.ToUpper();
var sendBuffer = Encoding.ASCII.GetBytes(result);
// gửi kết quả lại cho client
socket.SendTo(sendBuffer, remoteEndpoint);
```

Giao thức UDP (cũng như các giao thức truyền thông khác) đều chỉ chấp nhận và truyền đi các chuỗi byte mà không quan tâm đến ý nghĩa của dữ liệu. Việc đặt ra ý nghĩa của dữ liệu và chuyển đổi dữ liệu là trách nhiệm của ứng dụng. Vì lý do trên, trước khi gửi dữ liệu qua socket UDP (và cả socket TCP sẽ xem xét ở phần tiếp theo), chúng ta phải chuyển đổi dữ liệu người dùng thành một chuỗi byte.

Dữ liệu được xử lý trong chương trình C# rất đa dạng, bao gồm các kiểu dữ liệu cơ sở do C# định nghĩa sẵn (như số nguyên, số thực), chuỗi ký tự, ký tự, logic, enum, struct, class. Ứng với mỗi kiểu dữ liệu sẽ có cách thức khác nhau để chuyển đổi sang mảng byte. Trong đa số các trường hợp, việc chuyển đổi từ các kiểu dữ liệu của C# sang mảng byte là phức tạp. Việc chuyển đổi này có tên gọi là data serialization.

Đối với các kiểu dữ liệu cơ sở, C# hỗ trợ sẵn một số phương thức để thực hiện chuyển đổi sang mảng byte. Đối với dữ liệu văn bản, .NET cung cấp lớp Encoding với phương thức GetBytes. Đối với các kiểu dữ liệu cơ sở khác (số nguyên, số thực, logic), .NET cung cấp lớp BitConverter. Cả hai class này sẽ được xem xét chi tiết ở một bài khác.

Trong phần thực hành trên chúng ta chúng ta đã sử dụng lớp Encoding để chuyển đổi một chuỗi ký tự thành mảng byte theo mã hóa ASCII.

Khi muốn gửi dữ liệu qua socket UDP, chúng ta phải biết được địa chỉ của máy đích và số cổng mà tiến trình đích đang chờ. Để ra lệnh cho dịch vụ UDP bắt đầu gửi dữ liệu, chúng ta sử dụng lệnh SendTo của lớp Socket. Lệnh SendTo nhận một chuỗi byte (kết quả của biến đổi dữ liệu) và địa chỉ của tiến trình đích (lưu trong một đối tượng thuộc kiểu IPEndPoint).

```
// gửi mảng byte trên đến tiến trình server
socket.SendTo(sendBuffer, serverEndpoint);
// gửi kết quả lại cho client
socket.SendTo(sendBuffer, remoteEndpoint);
```

Khi phát lệnh này, chương trình UDP của hệ thống sẽ nhận chuỗi byte từ ứng dụng, đặt vào trước chuỗi byte đó một chuỗi byte nhỏ khác (gọi là header của UDP, sẽ xem xét chi tiết ở bài tiếp theo) để tạo ra một gói tin (thực chất chỉ là một chuỗi byte lớn) gọi là UDP datagram. Sau đó, UDP tiếp tục truyền chuỗi byte datagram này tới chương trình IP. Phương thức SendTo trả lại kết quả là tổng số byte đã được gửi đi.

Trong phần thực hành trên, Client yêu cầu người dùng tự nhập IP và port của Server (ở dạng chuỗi ký tự và chuyển đổi thành kiểu phù hợp), sau đó sử dụng thông tin này để gửi chuỗi byte đến tiến trình Server.

Đối với Server, để gửi dữ liệu (đã xử lý) lại cho Client, Server phải biết được endpoint của Client tương ứng. Giá trị endpoint này Server thu được khi nó nhận dữ liệu từ Client.

Nhận dữ liệu qua socket UDP

Việc nhận dữ liệu qua socket UDP phức tạp hơn so với gửi dữ liệu.

Khi một gói tin truyền qua socket UDP (dưới dạng một chuỗi byte), chúng ta không biết được chính xác nó chứa bao nhiêu byte. Vì vậy trước hết chúng ta phải tạo ra một mảng byte đệm đủ lớn để có thể chứa hết dữ liệu trong UDP datagram. Sau đó chúng ta gọi lệnh ReceiveFrom của lớp Socket để chuyển hết dữ liệu đọc được từ UDP socket sang mảng byte đệm này. Chuỗi byte trong mảng đệm sau đó sẽ được chuyển đổi thành dữ liệu người dùng mà chương trình có thể xử lý.

Trong code nhận dữ liệu của Server:

```
var size = 1024;
var receiveBuffer = new byte[size];
// biến này về sau sẽ chứa địa chỉ của tiến trình client nào gửi gói tin tới
```

```
EndPoint remoteEndpoint = new IPEndPoint(IPAddress.Any, 0);  
// khi nhận được gói tin nào sẽ lưu lại địa chỉ của tiến trình client  
var length = socket.ReceiveFrom(receiveBuffer, ref remoteEndpoint);  
var text = Encoding.ASCII.GetString(receiveBuffer, 0, length);
```

Ở đây chúng ta tạo ra một mảng đệm chứa tối đa 1024 byte. Nếu gói tin từ client vượt quá kích thước này, chúng ta sẽ mất những byte từ vị trí 1024. Cũng vì lý do này, client không nên gửi cho server các gói tin vượt quá 1024 byte.

Khi gọi lệnh ReceiveFrom, chúng ta phải cung cấp một tham số thứ hai theo kiểu ref (reference – truyền tham biến). Tham số thứ hai này sẽ chứa endpoint của máy vừa gửi gói tin tới. Đối với Server, đây chính là địa chỉ của tiến trình Client mà nó đang tương tác cùng. Server sẽ sử dụng endpoint này để gửi dữ liệu ngược trở lại cho Client.

Code nhận dữ liệu của Client cũng tương tự như vậy:

```
var size = 1024; // kích thước của bộ đệm  
var receiveBuffer = new byte[size]; // mảng byte làm bộ đệm  
// endpoint này chỉ dùng khi nhận dữ liệu  
EndPoint dummyEndpoint = new IPEndPoint(IPAddress.Any, 0);  
// nhận mảng byte từ dịch vụ UDP và lưu vào bộ đệm  
// biến dummyEndpoint có nhiệm vụ lưu lại địa chỉ của tiến trình nguồn  
// tuy nhiên, ở đây chúng ta đã biết tiến trình nguồn là Server  
// do đó dummyEndpoint không có giá trị sử dụng  
var length = socket.ReceiveFrom(receiveBuffer, ref dummyEndpoint);  
// chuyển đổi mảng byte về chuỗi  
var result = Encoding.ASCII.GetString(receiveBuffer, 0, length);
```

Ở đây chúng ta rất dễ nhận thấy, biến dummyEndpoint thực chất là dư thừa, bởi vì chúng ta đã biết chính xác endpoint của Server rồi. Việc sử dụng nó ở đây chỉ là để phù hợp với yêu cầu của phương thức ReceiveFrom.

Kết quả của phương thức ReceiveFrom là tổng số byte chuyển từ socket sang bộ đệm. Đây chính là kích thước thật của dữ liệu nhận được. Giá trị này có vai trò quan trọng trong việc biến đổi mảng byte trở lại dữ liệu người dùng. Trong cả code Client và Server, chúng ta đều dùng giá trị này cùng với phương thức GetString của của lớp Encoding để chuyển đổi mảng byte trở lại chuỗi ký tự ASCII.

Đặc điểm của truyền dữ liệu qua UDP socket

Có một số vấn đề trong gửi nhận dữ liệu cần lưu ý.

Khi phát lệnh gửi hoặc nhận dữ liệu, tiến trình sẽ bị “treo” tạm thời cho đến khi quá trình gửi hoặc nhận kết thúc. Đây là do chế độ blocking của socket, vốn được sử dụng mặc định.

Client và Server phải thỏa thuận với nhau về trình tự gửi – nhận dữ liệu. Nếu một bên phát lệnh gửi thì bên kia phải phát lệnh nhận. Nếu hai bên không thực hiện theo thỏa thuận này, ví dụ cùng phát lệnh nhận dữ liệu, cả hai sẽ cùng bị treo (dead blocking).

Vấn đề này có liên quan tới chế độ blocking của socket: khi phát lệnh nhận dữ liệu, lệnh này sẽ khóa luồng thực thi của ứng dụng lại để chờ dữ liệu đến. Chừng nào chưa có dữ liệu tới sẽ tiếp tục block việc thực thi của chương trình. Nếu cả hai tiến trình cùng phát lệnh nhận dữ liệu, cả hai sẽ cùng bị block vĩnh viễn vì không có dữ liệu tới.

Vì server phải hoạt động liên tục để phục vụ client, thông thường người ta sẽ đặt code của server vào một vòng lặp vô tận.

Nếu máy nguồn phát đi bao nhiêu gói tin (số lần gọi lệnh SendTo) thì máy đích phải có bấy nhiêu lệnh nhận gói tin. Nếu máy đích không nhận kịp thời, gói tin sẽ mất đi.

Đóng socket

Khi hoàn thành các thao tác gửi nhận dữ liệu và không có nhu cầu sử dụng socket nữa chúng ta có thể đóng socket đó lại bằng lệnh Close:

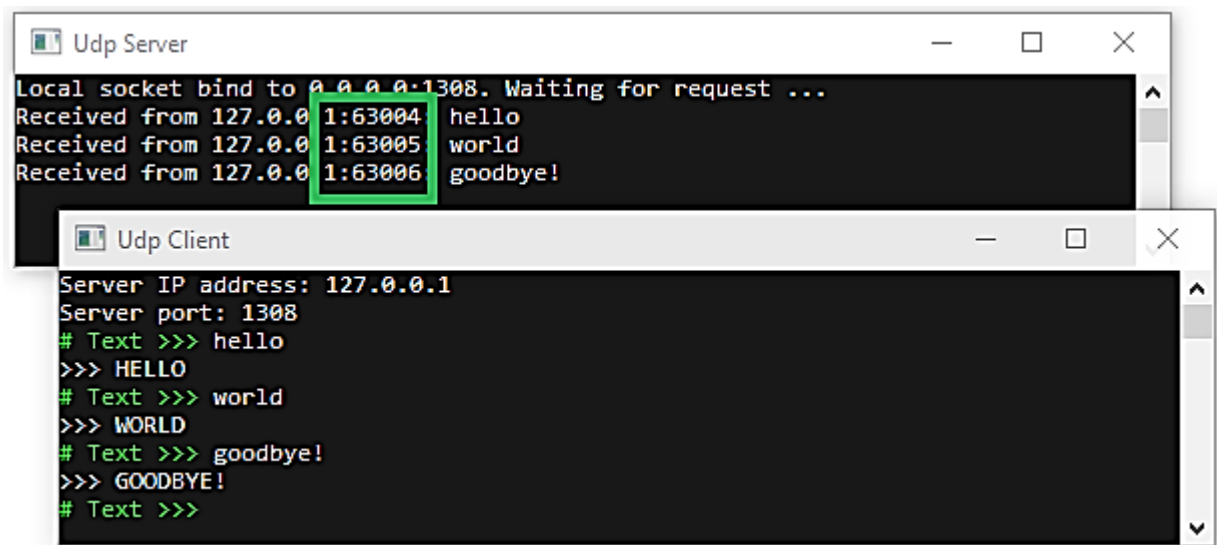
```
// đóng socket và giải phóng tài nguyên  
socket.Close();
```

Thao tác này sẽ giải phóng biến socket và tất cả các tài nguyên bị nó chiếm dụng.

Một khi gọi lệnh Close, socket sẽ không thể tái sử dụng được nữa. Nếu muốn sử dụng lại biến socket chúng ta bắt buộc phải khởi tạo nó sử dụng kỹ thuật khởi tạo đã trình bày ở trên.

Trong phần thực hành, mỗi khi Client hoàn thành một vòng truy vấn – phản hồi, chúng ta giải phóng biến socket. Khi người dùng quyết định gửi một chuỗi ký tự mới tới Server, chương trình Client khởi tạo một giá trị mới cho biến socket. Ở chương trình Server chúng ta sử dụng một đối tượng socket duy nhất và không đóng nó.

Chúng ta để ý rằng, mỗi lần Server nhận một chuỗi ký tự từ Client, nó đồng thời hiển thị địa chỉ của tiến trình Client, bao gồm IP và số cổng. Giá trị cổng của tiến trình Client thay đổi qua mỗi truy vấn. Đây là do mỗi lần khởi tạo đối tượng của lớp Socket, hệ thống lại cấp cho tiến trình “mượn” tạm một giá trị cổng UDP để sử dụng. Khi có lệnh Close (hoặc khi kết thúc chương trình), cổng này sẽ bị thu hồi và có thể lại được cấp cho một socket khác.



Hình 0.4. Biến đổi của cổng UDP sau mỗi lần tạo socket mới để gửi truy vấn

Việc khởi tạo hoặc đóng socket đều đòi hỏi thời gian xử lý. Do đó, cần cân đối giữa tái sử dụng một đối tượng socket hoặc giải phóng/khởi tạo lại socket mới. Nếu ứng dụng không có nhu cầu gửi nhận gói tin thường xuyên với tần suất cao thì nên giải phóng socket sau khi hoàn thành mỗi lượt thực hiện công việc. Ngược lại, chúng ta nên tái sử dụng đối tượng của socket.

c. Truyền dữ liệu với giao thức UDP

Truyền tải dữ liệu qua UDP không tạo liên kết ảo giữa các tiến trình tham gia truyền thông. Do không tạo ra liên kết ảo trước khi truyền dữ liệu, giao thức UDP có thể truyền dữ liệu đi ngay lập tức mà không cần thực hiện quá trình xây dựng liên kết phức tạp. Vì lý do này mà chỉ khi nào nhận được dữ liệu (ReceiveFrom), tiến trình mới biết được nó đang trao đổi dữ liệu với tiến trình nào (thông qua tham số thứ hai kiểu EndPoint). Cũng vì lý do này, UDP được gọi là giao thức phi liên kết, socket UDP còn được gọi là socket phi liên kết (connectionless socket).

Giao thức UDP đơn giản và cho phép truyền dữ liệu theo thời gian thực. Tuy nhiên, do thiếu các cơ chế kiểm soát dữ liệu trong quá trình truyền, giao thức UDP chỉ thích hợp khi truyền các loại dữ liệu không có yêu cầu cao về kiểm tra và sửa lỗi. Vì vậy UDP được sử dụng phổ biến nhất trong việc truyền tải dữ liệu đa phương tiện (video, audio, image), trong đó, việc mất một số datagram không ảnh hưởng đến việc sử dụng dữ liệu, đồng thời cần tốc độ truyền tải cao.

Truyền tải dữ liệu qua UDP không sử dụng các bộ nhớ đệm, do đó chương trình đích bắt buộc phải nhận gói tin UDP kịp thời. Nếu không phát lệnh nhận (ReceiveFrom) kịp thời, gói tin UDP sẽ mất đi. Vì server phải hoạt động liên tục để phục vụ client, thông thường người ta sẽ đặt code của server vào một vòng lặp vô tận. Nếu máy nguồn phát đi bao nhiêu gói tin (số lần gọi lệnh SendTo) thì máy đích phải có bấy nhiêu lệnh nhận gói tin. Nếu máy đích không nhận kịp thời, gói tin sẽ mất đi.

Khi phát lệnh gửi hoặc nhận dữ liệu, tiến trình sẽ bị “treo” tạm thời cho đến khi quá trình gửi hoặc nhận kết thúc. Đây là do chế độ blocking của socket, vốn được sử dụng mặc định. Do đó, Client và Server phải thỏa thuận với nhau về trình tự gửi – nhận dữ liệu. Nếu một bên phát lệnh gửi thì bên kia phải phát lệnh nhận. Nếu hai bên không thực hiện theo thỏa thuận này, ví dụ cùng phát lệnh nhận dữ liệu, cả hai sẽ cùng bị treo (dead blocking). Vấn đề này có liên quan tới chế độ blocking của socket: khi phát lệnh nhận dữ liệu, lệnh này sẽ khóa luồng thực thi của ứng dụng lại để chờ dữ liệu đến. Chừng nào chưa có dữ liệu tới sẽ tiếp tục block việc thực thi của chương trình. Nếu cả hai tiến trình cùng phát lệnh nhận dữ liệu, cả hai sẽ cùng bị block vĩnh viễn vì không có dữ liệu tới.

8.3. Lập trình với socket TCP

a. Thực hành

Tương tự như đối với socket UDP, chúng ta cũng bắt đầu tiếp xúc với socket TCP thông qua thực hiện một bài thực hành. Bài thực hành này cũng có chung yêu cầu với bài thực hành đã thực hiện đối với socket UDP: xây dựng một bộ ứng dụng theo mô hình client/server cho phép biến đổi một chuỗi ký tự người dùng nhập thành dạng chữ in hoa.

Bước 1. Tạo một Solution trống đặt tên là TcpSocket

Bước 2. Xây dựng lần lượt hai dự án thuộc kiểu Console App trong solution trên với tên gọi lần lượt là Server và Client.

Bước 3. Thiết lập để có thể debug đồng thời cả hai chương trình, trong đó Server sẽ khởi động trước.

Bước 4. Mở file Program.cs của Client và viết code như sau:

Client \ Program.cs

```
using System;  
using System.Net;
```

```

using System.Net.Sockets;
using System.Text;
namespace Client;
internal class Program {
    private static void Main(string[] args) {
        Console.Title = "TCP Client";
        // yêu cầu người dùng nhập ip của server
        Console.Write("Server IP address: ");
        var serverIpStr = Console.ReadLine();
        // chuyển đổi chuỗi ký tự thành object thuộc kiểu IPAddress
        var serverIp = IPAddress.Parse(serverIpStr);
        // yêu cầu người dùng nhập cổng của server
        Console.Write("Server port: ");
        var serverPortStr = Console.ReadLine();
        // chuyển chuỗi ký tự thành biến kiểu int
        var serverPort = int.Parse(serverPortStr);
        // đây là "địa chỉ" của tiến trình server trên mạng
        // mỗi endpoint chứa ip của host và port của tiến trình
        var serverEndpoint = new IPEndPoint(serverIp, serverPort);
        var size = 1024; // kích thước của bộ đệm
        var receiveBuffer = new byte[size]; // mảng byte làm bộ đệm
        while (true) {
            // yêu cầu người dùng nhập một chuỗi
            Console.ForegroundColor = ConsoleColor.Green;
            Console.Write("# Text >>> ");
            Console.ResetColor();
            var text = Console.ReadLine();
            // khởi tạo object của lớp socket để sử dụng dịch vụ TCP
            // lưu ý SocketType của TCP là Stream
            var socket = new Socket(SocketType.Stream, ProtocolType.TCP);
            // tạo kết nối tới Server
            socket.Connect(serverEndpoint);
            // biến đổi chuỗi thành mảng byte
            var sendBuffer = Encoding.ASCII.GetBytes(text);
            // gửi mảng byte trên đến tiến trình server
            socket.Send(sendBuffer);
            // không tiếp tục gửi dữ liệu nữa
            socket.Shutdown(SocketShutdown.Send);
            // nhận mảng byte từ dịch vụ TCP và lưu vào bộ đệm
            var length = socket.Receive(receiveBuffer);
            // chuyển đổi mảng byte về chuỗi
            var result = Encoding.ASCII.GetString(receiveBuffer, 0, length);
            // xóa bộ đệm (để lần sau sử dụng cho yên tâm)
            Array.Clear(receiveBuffer, 0, size);
            // không tiếp tục nhận dữ liệu nữa
            socket.Shutdown(SocketShutdown.Receive);
            // đóng socket và giải phóng tài nguyên
            socket.Close();
            // in kết quả ra màn hình
            Console.WriteLine($">>> {result}");
        }
    }
}

```

Chúng ta có thể để ý, code của cả Client và Server đều tái sử dụng phần khung của nội dung thực hành 1 UDP nhưng thay thế phần truyền thông UDP bằng TCP. So sánh với code của thực hành 1 UDP để thấy được sự khác biệt.

Bước 5. Mở file Program.cs của Server và viết code như sau:

```

using System;
using System.Net;
using System.Net.Sockets;
using System.Text;
namespace Server
{
    internal class Program
    {
        private static void Main(string[] args)
        {
            Console.Title = "TCP Server";
            // giá trị Any của IPAddress tương ứng với IP của tất cả các giao diện mạng trên máy
            var localIp = IPAddress.Any;
            // tiến trình server sẽ sử dụng cổng tcp 1308
            var localPort = 1308;
            // biến này sẽ chứa "địa chỉ" của tiến trình server trên mạng
            var localEndPoint = new IPEndPoint(localIp, localPort);
            // tcp sử dụng đồng thời hai socket:
            // một socket để chờ nghe kết nối, một socket để gửi/nhận dữ liệu
            // socket listener này chỉ làm nhiệm vụ chờ kết nối từ Client
            var listener = new Socket(AddressFamily.InterNetwork, SocketType.Stream,
ProtocolType.TCP);
            // yêu cầu hệ điều hành cho phép chiếm dụng cổng tcp 1308
            // server sẽ nghe trên tất cả các mạng mà máy tính này kết nối tới
            // chỉ cần gói tin tcp đến cổng 1308, tiến trình server sẽ nhận được
            listener.Bind(localEndPoint);
            // bắt đầu lắng nghe chờ các gói tin tcp đến cổng 1308
            listener.Listen(10);
            Console.WriteLine($"Local socket bind to {localEndPoint}. Waiting for
request ...");
            var size = 1024;
            var receiveBuffer = new byte[size];
            while (true)
            {
                // tcp đòi hỏi một socket thứ hai làm nhiệm vụ gửi/nhận dữ liệu
                // socket này được tạo ra bởi lệnh Accept
                var socket = listener.Accept();
                Console.WriteLine($"Accepted connection from {socket.RemoteEndPoint}");
                // nhận dữ liệu vào buffer
                var length = socket.Receive(receiveBuffer);
                // không tiếp tục nhận dữ liệu nữa
                socket.Shutdown(SocketShutdown.Receive);
                var text = Encoding.ASCII.GetString(receiveBuffer, 0, length);
                Console.WriteLine($"Received: {text}");
                // chuyển chuỗi thành dạng in hoa
                var result = text.ToUpper();
                var sendBuffer = Encoding.ASCII.GetBytes(result);
                // gửi kết quả lại cho client
                socket.Send(sendBuffer);
                Console.WriteLine($"Sent: {result}");
                // không tiếp tục gửi dữ liệu nữa
                socket.Shutdown(SocketShutdown.Send);
                // đóng kết nối và giải phóng tài nguyên
                Console.WriteLine($"Closing connection from
{socket.RemoteEndPoint}\r\n");
                socket.Close();
                Array.Clear(receiveBuffer, 0, size);
            }
        }
    }
}

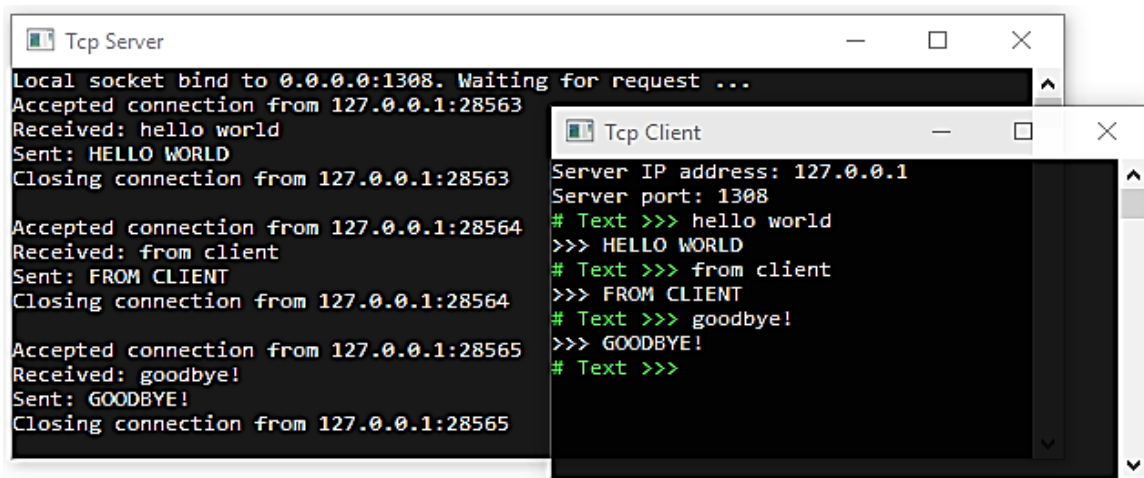
```

```
}
}
}
```

Bước 6. Dịch và chạy thử hệ thống, kết quả thu được như Hình 0.1.

Khi khởi động chương trình client sẽ yêu cầu nhập địa chỉ IP và số cổng của tiến trình server. Nếu kết nối thành công chúng ta có thể bắt đầu gửi dữ liệu. Nhập một chuỗi ký tự bất kỳ vào dấu nhắc lệnh. Chuỗi ký tự đó sẽ được chuyển sang server. Server biến chuỗi thành chuỗi in hoa và gửi lại client. Client hiển thị kết quả.

Chương trình Client về hình thức bề ngoài không có gì khác biệt so với chương trình viết ở phần thực hành 1 của UDP. Tuy nhiên, phần truyền thông đã thay đổi hoàn toàn.



Hình 0.1. Hoạt động của chương trình TCP

Trong phần tiếp theo chúng ta sẽ phân tích code của Client và Server để hiểu được cách lập trình với socket TCP.

b. Kỹ thuật lập trình TCP socket với C#

Trong phần này chúng ta sẽ phân tích chi tiết cách thức lập trình TCP socket trong C#. Các phân tích này dựa trên code của bài thực hành lập trình tcp socket với C# cơ bản chúng ta đã thực hiện.

Khởi tạo socket TCP

Để khởi tạo socket TCP chúng ta vẫn áp dụng một trong hai overload của hàm tạo lớp Socket, tuy nhiên phải thay đổi giá trị của SocketType và ProtocolType cho phù hợp với TCP. Socket TCP yêu cầu SocketType là Stream, và ProtocolType là TCP.

Ở Client chúng ta đã sử dụng cách khởi tạo như sau:

```
// khởi tạo object của lớp socket để sử dụng dịch vụ TCP
// lưu ý SocketType của TCP là Stream
var socket = new Socket(SocketType.Stream, ProtocolType.TCP);
```

Chúng ta cũng để ý rằng, đối với Client chúng ta chỉ khởi tạo một biến Socket. Đối với Server chúng ta phải khởi tạo hai biến Socket: Biến thứ nhất đặt tên là listener và được khởi tạo bằng hàm tạo của lớp Socket; Biến thứ hai đặt tên là socket và nó được khởi tạo bằng phương thức Accept của listener.

```
// tcp đòi hỏi một socket thứ hai làm nhiệm vụ gửi/nhận dữ liệu
// socket này được tạo ra bởi lệnh Accept
var socket = listener.Accept();
```

Ở Client chỉ cần sử dụng một socket để việc gửi và nhận dữ liệu. Tuy nhiên, ở Server phải một socket chuyên môn đảm nhiệm việc chờ nghe yêu cầu kết nối từ client. Một khi có yêu cầu kết nối tới, Server tạo một socket khác để gửi và nhận dữ liệu.

Chính socket thứ hai của Server mới là “đầu cầu” kết nối với socket của Client. Một khi thiết lập xong đầu cầu này của liên kết, socket listener sẽ quay về nhiệm vụ chính của mình: tiếp tục chờ yêu cầu liên kết mới từ các client khác.

Liên kết Server với cổng và lắng nghe

Tương tự như với socket UDP, socket listener của Server sau khi khởi tạo cũng phải yêu cầu hệ thống cho phép chiếm dụng một cổng đang trống bằng lệnh Bind:

```
// yêu cầu hệ điều hành cho phép chiếm dụng cổng tcp 1308
// server sẽ nghe trên tất cả các mạng mà máy tính này kết nối tới
// chỉ cần gói tin tcp đến cổng 1308, tiến trình server sẽ nhận được
listener.Bind(localEndPoint);
```

Ở đây cần lưu ý rằng, cổng chiếm dụng bởi TCP không liên quan tới cổng bị chiếm dụng bởi UDP. Điều này có nghĩa là, một tiến trình sử dụng UDP đã chiếm cổng, ví dụ, 1308, thì một tiến trình khác sử dụng TCP cũng vẫn có thể đồng thời chiếm dụng cổng 1308.

Vì vậy, để phân biệt rõ, người ta cũng gọi là cổng TCP 1308, cổng UDP 1308. Để thử nghiệm, chúng ta có thể chạy thử đồng thời chương trình Server UDP và Server TCP. Cả hai tiến trình này đều sử dụng cổng 1308 nhưng của hai giao thức khác nhau.

Khác với Server UDP, Server TCP còn phải thực hiện một thao tác nữa trước khi có thể tiếp nhận kết nối:

```
// bắt đầu lắng nghe chờ các gói tin tcp đến cổng 1308
listener.Listen(10);
```

Lệnh Listen đặt socket listener vào chế độ chờ nghe yêu cầu kết nối từ các client. Socket listener cũng có thể xem là nơi thực hiện quá trình bắt tay ba bước. Khi quá trình bắt tay kết thúc, listener sinh ra một socket bản sao của nó để thực hiện gửi/nhận dữ liệu với socket của client. Khi hoàn thành nhiệm vụ gửi/nhận dữ liệu, socket bản sao sẽ bị hủy bỏ.

Khi sử dụng lệnh Listen phải cung cấp một tham số có tên gọi là backlog thuộc kiểu số nguyên. Backlog quy định số lượng tối đa yêu cầu liên kết được đặt trong hàng đợi chờ đến lượt xử lý.

Hình dung tại mỗi thời điểm có thể có nhiều yêu cầu kết nối từ các client nhưng Server chỉ có thể xử lý một yêu cầu (thực hiện bắt tay ba bước với client đó). Các yêu cầu còn lại sẽ được đặt trong một hàng đợi để chờ đến lượt xử lý. Số lượng yêu cầu tối đa đặt trong hàng chờ được quy định bởi tham số backlog.

Tạo kết nối TCP

Theo quy trình làm việc của TCP, trước khi có thể trao đổi dữ liệu, client phải thực hiện kết nối tới server. Quá trình tạo kết nối này đã được mô tả chi tiết trong bài lý thuyết về giao thức TCP.

Đối với Client, chúng ta sử dụng lệnh Connect của lớp Socket:

```
// tạo kết nối tới Server  
socket.Connect(serverEndpoint);
```

Quá trình tạo liên kết này chỉ có thể theo chiều từ Client tới Server: Client đưa ra yêu cầu kết nối, Server lắng nghe và xử lý yêu cầu kết nối từ Client.

Để Server bắt đầu xử lý yêu cầu kết nối từ Client, chúng ta sử dụng lệnh Accept của lớp Socket. Khi quá trình xử lý yêu cầu kết nối kết thúc và kết nối TCP được tạo ra, một socket mới được Server sinh ra để thực hiện gửi/nhận dữ liệu.

```
// tcp đòi hỏi một socket thứ hai làm nhiệm vụ gửi/nhận dữ liệu  
// socket này được tạo ra bởi lệnh Accept  
var socket = listener.Accept();
```

Gửi dữ liệu qua liên kết TCP

Tương tự như UDP, các phương thức gửi và nhận dữ liệu của socket TCP cũng chỉ làm việc với các chuỗi byte. Nhiệm vụ chuyển đổi dữ liệu của ứng dụng thành chuỗi byte (để gửi đi) và chuyển đổi chuỗi byte nhận được thành dữ liệu thuộc về chương trình.

Socket TCP sử dụng một phương thức khác để gửi dữ liệu. Ở Client gửi dữ liệu theo cách sau:

```
// biến đổi chuỗi thành mảng byte  
var sendBuffer = Encoding.ASCII.GetBytes(text);  
// gửi mảng byte trên đến tiến trình server  
socket.Send(sendBuffer);
```

Tương tự như vậy, ở Server cũng sử dụng phương thức Send.

```
var sendBuffer = Encoding.ASCII.GetBytes(result);  
// gửi kết quả lại cho client  
socket.Send(sendBuffer);
```

Chúng ta thấy lệnh Send được sử dụng để gửi dữ liệu với socket TCP. Thông tin về tiến trình đối tác đã được lưu sẵn trong socket sau khi hoàn tất quá trình thiết lập liên kết TCP, do đó chúng ta không cần cung cấp thông tin như đối với socket UDP.

Đặc điểm truyền dữ liệu qua socket TCP

Như nội dung “truyền dữ liệu” của giao thức TCP đã trình bày, khi phát lệnh Send, TCP không đóng gói và truyền dữ liệu đi ngay lập tức. Thay vào đó, chuỗi byte dữ liệu sẽ được đưa vào một bộ nhớ đệm của TCP. Sau đó, chương trình TCP sẽ độc lập xử lý việc lấy dữ liệu từ bộ nhớ đệm, phân mảnh, đóng gói và chuyển tiếp xuống chương trình IP. Toàn bộ quá trình phức tạp như truyền segment, nhận ACK, kiểm soát giá trị các trường của header được TCP thực hiện mà không có sự can thiệp của ứng dụng.

Bởi vì dữ liệu trước hết được đưa vào bộ đệm, quá trình truyền dữ liệu của TCP khi nhìn từ ứng dụng sẽ giống như truyền một luồng byte liên tục chứ không phải ở dạng các gói byte độc lập như của UDP. Trên thực tế, .NET coi việc truyền dữ liệu qua liên kết TCP là một luồng dữ liệu (Stream), coi TCP socket là một nguồn dữ liệu (tương tự như file), và cung cấp lớp NetworkStream chuyên để làm việc với loại luồng dữ liệu này.

Cơ chế truyền dữ liệu của TCP khuyến nghị không nên phát nhiều lệnh Send với các dữ liệu nhỏ. Ngược lại, nên chuẩn bị một mảng dữ liệu lớn và phát một lệnh Send. Khi đó, cơ chế phân mảnh dữ liệu của TCP sẽ hỗ trợ tốt hơn cho việc truyền dữ liệu.

Vì được hình dung như một dòng byte liên tục, kể cả khi phát nhiều lệnh Send, dữ liệu trong mỗi lệnh Send cũng không được đóng gói độc lập như UDP và do đó ranh giới của dữ liệu ứng dụng không được duy trì. Chương trình ứng dụng phải có nhiệm vụ tạo ra các ranh giới để chương trình đích có thể tự phân tách các phần của dữ liệu.

Luồng dữ liệu, lớp NetworkStream và Data Serialization sẽ được xem xét chi tiết ở chương sau.

Shutdown tcp socket

Ở cả Client và Server chúng ta thấy rằng sau lệnh Send đều có thêm một lệnh ShutDown.

```
// không tiếp tục gửi dữ liệu nữa  
socket.Shutdown(SocketShutdown.Send);
```

Phương thức ShutDown của lớp Socket chỉ sử dụng với socket TCP và có ảnh hưởng quan trọng tới quá trình gửi/nhận dữ liệu.

Phương thức này nhận một tham số kiểu enum SocketShutdown với ba giá trị: Send, Receive, Both.

Lệnh ShutDown với giá trị tham số SocketShutdown.Send không ngắt kết nối TCP. Nó ngăn không cho thực hiện các lệnh gửi dữ liệu tiếp theo. Nếu còn dữ liệu trong buffer chưa gửi hết, TCP sẽ cố gắng gửi hết dữ liệu sau đó phát đi một chuỗi byte đặc biệt tới socket đối tác báo rằng quá trình truyền dữ liệu đã kết thúc. Đây là tín hiệu báo kết thúc dòng byte dữ liệu.

Ở máy nhận, lệnh nhận dữ liệu (sẽ xem ở phần dưới) sẽ trả về giá trị 0, báo rằng dòng byte dữ liệu nhận đã kết thúc. Tín hiệu này có vai trò quan trọng trong việc gửi/nhận dữ liệu kích thước lớn.

Nhận dữ liệu

Để đọc dữ liệu vào chương trình chúng ta sử dụng phương thức Receive của lớp Socket như sau:

```
// nhận mảng byte từ dịch vụ TCP và lưu vào bộ đệm  
var length = socket.Receive(receiveBuffer);  
// chuyển đổi mảng byte về chuỗi  
var result = Encoding.ASCII.GetString(receiveBuffer, 0, length);  
// không tiếp tục nhận dữ liệu nữa  
socket.Shutdown(SocketShutdown.Receive);
```

Tương tự như với UDP, lệnh đọc này cũng copy dữ liệu vào một mảng đệm của chương trình để xử lý sau. Lệnh Receive cũng trả về số lượng byte đọc được từ TCP buffer.

Tuy nhiên, việc nhận dữ liệu với socket TCP khác biệt rất lớn với UDP. Khi dữ liệu tới, TCP sẽ lưu hết vào một bộ nhớ đệm để chờ chương trình đọc ra. Khi nào chương trình phát lệnh đọc, dữ liệu sẽ được chuyển từ bộ nhớ đệm này vào chương trình và phản tương ứng trong bộ nhớ đệm sẽ được giải phóng.

Nếu một bên kết thúc quá trình gửi dữ liệu và gọi lệnh ShutDown với tham số Send, lệnh Receive sẽ trả về giá trị 0, báo hiệu không còn dữ liệu để nhận và dòng byte nhận đã chấm dứt. Giá trị này có ý nghĩa quan trọng nếu nhận một khối lượng dữ liệu lớn vượt quá kích thước của mảng đệm. Khi đó, việc đọc dữ liệu phải đặt trong một vòng lặp và thực hiện liên tục cho tới khi Receive trả về giá trị 0.

Sau khi hoàn tất nhận dữ liệu, chúng ta có thể gọi lệnh ShutDown với tham số Receive để ngăn các thao tác đọc dữ liệu tiếp theo. Nếu có dữ liệu tiếp tục đến cũng sẽ không được tiếp nhận nữa.

Do TCP đưa dữ liệu nhận được vào bộ đệm, nếu chương trình không kịp đọc và xử lý thì dữ liệu vẫn tiếp tục nằm ở đó chứ không mất đi. Đây là sự khác biệt rất lớn với UDP.

Tương tự như khi gửi, quá trình nhận dữ liệu từ TCP cũng được nhìn nhận như đọc một dòng byte liên tục và do đó có thể sử dụng giao diện luồng để hỗ trợ đọc dữ liệu.

Chúng ta nên lưu ý rằng, các lệnh Connect, Accept, Send và Receive đã gặp ở trên mặc định đều hoạt động ở chế độ blocking. Điều này có nghĩa là một khi gọi một trong các lệnh tương ứng, toàn bộ việc thực thi của các lệnh phía sau đều sẽ dừng lại cho đến khi lệnh này hoàn tất.

Ví dụ, đối với Client, lệnh Connect sẽ block luồng thực thi lại cho đến khi kết nối thành công, hoặc phát ra lỗi. Đối với Server, lệnh Accept sẽ block luồng thực thi cho đến khi có một yêu cầu kết nối tới và xử lý thành công. Chừng nào chưa có Client phát yêu cầu kết nối tới, Server sẽ bị block.

Chế độ blocking giúp việc lập trình mạng an toàn và đơn giản hơn vì quá trình xử lý thường phải thực hiện theo trật tự tuyến tính, bước sau chỉ thực hiện được khi bước trước thành công. Đây là chế độ hoạt động mặc định của socket trên windows.

Tuy nhiên, chế độ block cũng có nhiều hạn chế, đặc biệt với các ứng dụng có giao diện đồ họa và đối với chương trình server. Trong quá trình thực hiện các lệnh blocking, toàn bộ giao diện sẽ bị treo. Chế độ block làm giảm hiệu quả của Server khi phục vụ cùng lúc nhiều Client.

Ở chương sau, chúng ta sẽ xem xét một số phương pháp giải quyết các vấn đề bày thông qua lập trình bất đồng bộ, lập trình đa luồng, và sử dụng chế độ non-blocking.

Đóng liên kết

Một khi quá trình trao đổi dữ liệu kết thúc có thể đóng liên kết lại và giải phóng các tài nguyên liên quan. Để đóng một socket TCP sử dụng phương thức Close của lớp Socket:

```
// đóng socket và giải phóng tài nguyên  
socket.Close();
```

Trên thực tế, khi gọi lệnh Close, trước hết socket tự gọi lệnh ShutDown với giá trị tham số là Both để ngăn cản các thao tác đọc ghi. Lệnh đóng liên kết này sẽ kích hoạt quá trình như đã mô tả ở phần nội dung về giao thức TCP. Sau đó, đối tượng sẽ bị hủy bỏ. Sau lệnh này, biến socket sẽ không trở vào đối tượng nào nữa. Việc gọi các lệnh trên socket sau khi gọi lệnh Close đều gây lỗi.

Có một cách thức khác để ngắt liên kết nhưng không giải phóng đối tượng của socket là sử dụng phương thức Disconnect của lớp Socket. Phương thức Disconnect cũng gọi lệnh ShutDown như trên và ngắt kết nối nhưng không hủy bỏ đối tượng. Do đó, sau này có thể tái sử dụng đối tượng socket. Tuy nhiên, quá trình thiết lập liên kết sẽ phải thực hiện lại.

c. Một số đặc điểm của lập trình với socket TCP

Xử lý liên kết TCP trong chương trình

Khi phát triển ứng dụng mạng sử dụng socket TCP, chúng ta sẽ phải trả lời câu hỏi: sau khi hoàn thành việc gửi nhận dữ liệu, chúng ta muốn giữ lại liên kết này để tiếp tục sử dụng trong tương lai hay ngắt bỏ liên kết. Việc lựa chọn này ảnh hưởng lớn đến cách thức viết chương trình.

Điều này dẫn đến hai mô hình xử lý liên kết TCP.

Trong mô hình thứ nhất, liên kết TCP sẽ bị hủy bỏ ngay sau khi hoàn thành việc trao đổi dữ liệu. Trong mô hình truyền thông điệp kiểu truy vấn / phản hồi, đây là khi kết thúc một vòng truy vấn – phản hồi. Đối với vòng truy vấn tiếp theo, một liên kết TCP mới sẽ được thiết lập. Mô hình này cũng tạo ra một đặc điểm khác của việc truyền thông tin: tính không duy trì trạng thái (stateless).

Trong mô hình thứ hai, liên kết TCP sẽ được duy trì lâu dài để sử dụng trong nhiều vòng truy vấn – phản hồi cho đến khi một trong hai phía chính thức hủy bỏ nó. Mô hình này tạo ra tính duy trì trạng thái (stateful) của truyền thông.

Vấn đề nằm ở chỗ, việc hủy bỏ liên kết và tái thiết lập liên kết TCP đều mất nhiều thời gian. Tuy nhiên, việc duy trì liên kết TCP lại đòi hỏi tài nguyên. Đối với client, tài nguyên này có thể không đáng kể. Tuy nhiên, đối với server, khi phải đồng thời phục vụ rất nhiều client, việc duy trì nhiều liên kết TCP cùng lúc gây tốn kém rất nhiều tài nguyên, nhất là nếu như liên kết đó không được khai thác.

Trên thực tế, giao thức HTTP /1.1 mặc định sử dụng cơ chế ngắt kết nối TCP sau mỗi vòng truy vấn phản hồi. Trong khi đó kênh điều khiển của giao thức FTP lại duy trì kết nối TCP liên tục đến khi người dùng thoát hoặc không có dữ liệu trao đổi qua kênh điều khiển trong một khoảng thời gian nhất định.

Như vậy, không thể nói là mô hình nào có ưu thế hơn. Việc lựa chọn mô hình nào phụ thuộc nhiều vào đặc điểm và yêu cầu của phần mềm ứng dụng.

Truyền dữ liệu qua TCP

Khi tiến trình nguồn phát lệnh Send, thực tế dữ liệu không được lập tức đóng gói vào truyền đi. Thay vào đó, dữ liệu được chuyển tới một bộ nhớ đệm của TCP (TCP buffer), được hình thành trong quá trình bắt tay ba bước. Theo thời gian, TCP sẽ lấy dữ liệu từ bộ nhớ đệm đó, đóng gói thành segment, và chuyển lên đường truyền. Trong khoảng thời gian chờ ở bộ nhớ đệm, ứng dụng có thể gửi thêm dữ liệu và dữ liệu này sẽ tiếp tục được bổ sung vào bộ nhớ đệm. Như vậy, số segment tạo ra không tương đương với số lệnh Send được gọi. Đây là một điểm khác biệt với UDP.

Tương tự, TCP trên máy đích cũng sử dụng một bộ nhớ đệm cục bộ của riêng mình. Tất cả dữ liệu đi theo liên kết TCP tới máy đích sẽ được lưu trữ ở bộ nhớ đệm này. Một khi có dữ liệu trong bộ đệm, chương trình có thể đọc được bằng cách gọi phương thức Receive. Lệnh này sẽ di chuyển khối byte từ bộ đệm TCP vào chương trình và xóa bỏ khối byte tương ứng trong bộ đệm. Nếu chương trình đích chưa phát lệnh đọc dữ liệu thì dữ liệu vẫn tiếp tục nằm ở bộ nhớ đệm và không bị mất đi. Đây là điểm rất khác biệt với cách đọc dữ liệu của UDP.

Do đặc điểm trên, các lệnh Send trên máy nguồn và Receive trên máy đích không đồng bộ nhau. Nhiều lệnh Send gửi dữ liệu đi có thể chỉ cần 1 lệnh Receive để nhận dữ liệu về. Đây là điểm khác biệt rất lớn với UDP cần lưu ý.

Do sử dụng buffer, khi nhìn từ khía cạnh ứng dụng, liên kết và truyền dữ liệu qua TCP giống như một luồng byte liên tục chạy từ buffer của máy nguồn tới buffer của máy đích. Vì vậy, khi sử dụng TCP socket chúng ta có thể áp dụng kỹ thuật lập trình luồng dữ liệu với lớp NetworkStream. UDP không áp dụng kỹ thuật luồng dữ liệu.

Vì vận chuyển dữ liệu theo một dòng byte liên tục, TCP không đảm bảo duy trì “ranh giới” giữa các thông điệp. Việc phân tách nội dung thông điệp tầng ứng dụng thuộc trách nhiệm của chương trình ứng dụng. Đây cũng là một đặc điểm quan trọng cần lưu ý khi lập trình với TCP.

Cắt ghép dữ liệu

Khi TCP gửi đi dữ liệu kích thước lớn (ví dụ, một file video lớn), dữ liệu sẽ được tự động cắt thành từng phần để đóng gói vào các segment. Kích thước tối đa của segment MSS (Maximum Segment Size) được thiết lập tự động bằng cách xác định kích thước tối đa của frame dữ liệu của tầng liên kết. Việc thiết lập MSS như vậy đảm bảo cho TCP segment có thể nằm vừa khít trong một frame với kích thước đủ lớn để có hiệu suất gửi dữ liệu cao nhất. Mỗi segment đều được đánh số thứ tự, lưu trong trường Sequence Number của TCP header. Quá trình này được TCP làm tự động.

Tại máy đích, TCP căn cứ vào số thứ tự này để tự động ghép nối các segment trở lại theo đúng thứ tự chúng được cắt và gửi đi. Khi chương trình phát lệnh Receive sẽ luôn nhận được phần dữ liệu đã ghép nối hoàn chỉnh.

Đặc điểm này giúp chương trình ứng dụng có thể dễ dàng và an toàn gửi dữ liệu kích thước lớn qua TCP. Chương trình không cần thực hiện cắt ghép dữ liệu.

Thực tế, khi truyền dữ liệu kích thước lớn qua TCP socket, người ta thường sử dụng kỹ thuật luồng dữ liệu. Luồng dữ liệu cho phép sử dụng một bộ nhớ đệm nhỏ trong chương trình để trung chuyển dữ liệu, ví dụ từ file tới TCP buffer, hoặc từ TCP buffer về file. Kỹ thuật này cho phép truyền khối lượng lớn dữ liệu nhưng không cần sử dụng quá nhiều bộ nhớ tạm để lưu dữ liệu. Kỹ thuật lập trình với luồng dữ liệu sẽ được xem xét trong một bài học riêng.

8.4. Lập trình với giao thức HTTP

a. Xây dựng web client đơn giản

Trong bài thực hành này, chúng ta sẽ cùng xây dựng một web client đơn giản nhất. Web client này chỉ có thể gửi một truy vấn về server. Khi nhận phản hồi nó sẽ in kết quả ra giao diện console.

(1) Tạo dự án C# (Console App) và đặt tên cho dự án là HttpClient_Console.

(2) Mở file Program.cs và viết code như sau:

Program.cs

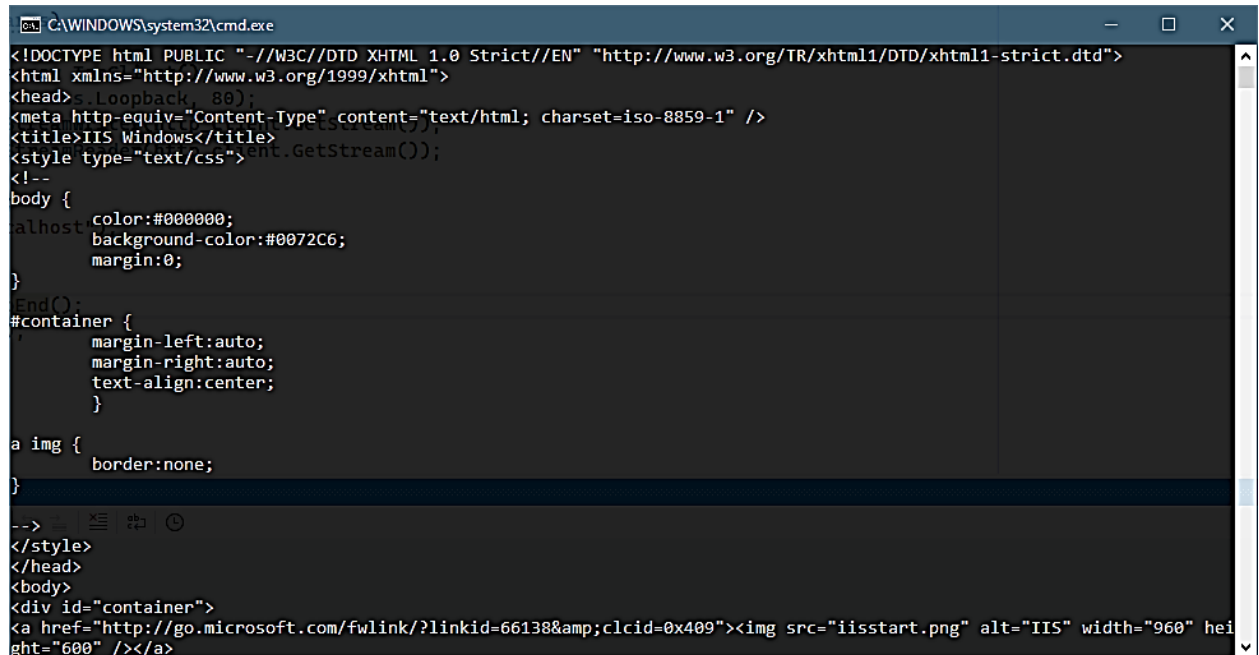
```
using System;
using System.Net;
using System.Net.Sockets;
using System.IO;
namespace HttpClient;
class Program {
    static void Main(string[] args) {
        TcpClient http_client = new TcpClient();
        http_client.Connect(IPAddress.Loopback, 80);
        StreamWriter sw = new StreamWriter(http_client.GetStream());
        var request = "GET /\r\nHost: localhost";
        sw.WriteLine(request);
        sw.Flush();
        StreamReader sr = new StreamReader(http_client.GetStream());
        var response = sr.ReadToEnd();
        Console.WriteLine(response);
    }
}
```

```

        sw.Close();
        sr.Close();
        Console.ReadKey();
    }
}

```

(3) Dịch và chạy chương trình. Giả sử bạn đang sử dụng server IIS, kết quả sẽ như sau:



Hình 0.1. Kết quả chạy chương trình HTTP client đơn giản

Đây là mã HTML mà chương trình chủ IIS gửi lại cho chương trình qua HTTP. Nếu sử dụng Apache, kết quả thu được có thể sẽ khác.

Trong ví dụ trên, chúng ta đã thực hiện các thao tác sau:

(1) Thiết lập một liên kết TCP tới cổng số 80 của máy tính cục bộ:

```

TcpClient http_client = new TcpClient();
http_client.Connect(IPAddress.Loopback, 80);

```

(2) Trên liên kết TCP này, chúng ta gửi đi một chuỗi văn bản theo cấu trúc của HTTP request:

```

StreamWriter sw = new StreamWriter(http_client.GetStream());
var request = "GET /\r\nHost: localhost";
sw.WriteLine(request);
sw.Flush();

```

Ở đây chúng ta sử dụng Stream Adapter StreamWriter để tiện lợi cho việc gửi từng dòng văn bản lên luồng mạng TCP.

(3) Sau khi gửi đi chuỗi truy vấn, chúng ta đọc toàn bộ phản hồi về chuỗi html và in ra màn hình console:

```

StreamReader sr = new StreamReader(http_client.GetStream());
var response = sr.ReadToEnd();

```

```
Console.WriteLine(response);
```

Ở đây, chúng ta sử dụng Stream Adapter StreamReader để tiện lợi cho việc đọc cả chuỗi văn bản lớn từ luồng mạng TCP.

Như vậy, chúng ta có thể thấy, chương trình do chúng ta viết đã có thể trao đổi thông tin với web server IIS / Apache thông qua giao thức HTTP.

Đây là ví dụ đơn giản nhất về việc tạo gói tin truy vấn HTTP và nhận phản hồi từ server.

Lưu ý, mặc dù bạn có thể tải nội dung trang web HTML về ứng dụng, bạn không thể hiển thị nó như trong trình duyệt. HTML bản chất chỉ là chuỗi ký tự và nó cần các cơ chế của trình duyệt để có thể hiển thị thành giao diện đồ họa.

b. Xây dựng web server đơn giản

Trong phần này, chúng ta sẽ cùng xây dựng một web server đơn giản. Web server này có thể nhận truy vấn từ trình duyệt thực sự (Chrome, Edge, Firefox, v.v.) và trả về một đoạn HTML.

(1) Tạo một dự án mới kiểu Console App, đặt tên là HttpServerSimple.

(2) Mở file Program.cs và viết code như sau:

Program.cs

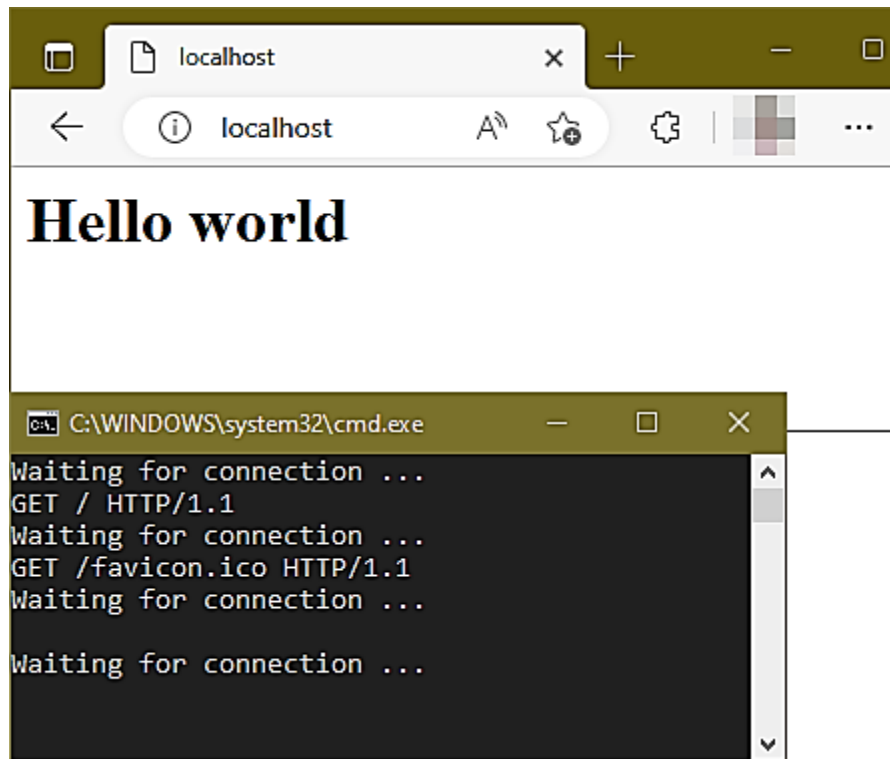
```
using System;
using System.Net;
using System.Net.Sockets;
using System.IO;
namespace HttpServerSimple;
class Program {
    static void Main(string[] args) {
        TcpListener listener = new TcpListener(IPAddress.Any, 80);
        listener.Start(5);
        while (true) {
            try {
                Console.WriteLine("Waiting for connection ...");
                var client = listener.AcceptTcpClient();
                var sr = new StreamReader(client.GetStream());

                string request = sr.ReadLine();
                Console.WriteLine(request);
                var sw = new StreamWriter(client.GetStream());
                sw.WriteLine("HTTP/1.1 200 OK\n");
                sw.WriteLine("<html><head></head><body><h1>Hello
world</h1><body></html>");
                sw.Flush();
                client.Close();
            } catch (Exception) { }
        }
    }
}
```

(3) Kiểm tra xem IIS hay Apache có hoạt động hay không. Nếu các chương trình này hoạt động thì cần tắt đi.

(4) Dịch và chạy thử chương trình.

(5) Mở một trình duyệt bất kỳ và gõ vào đường link: <http://localhost>. Bạn sẽ thu được kết quả như Hình 12.15.



Hình 0.2. Chương trình server cung cấp trang web cho trình duyệt

Như vậy chương trình server đơn giản bạn viết thực sự có thể tiếp nhận truy vấn từ trình duyệt và trả lại kết quả cho trình duyệt.

Trong ví dụ đơn giản trên, chúng ta đã thực hiện những thao tác sau:

(1) Khởi tạo một TCP socket gắn với mọi giao diện mạng trên máy cục bộ và lắng nghe cổng TCP/80

```
TcpListener listener = new TcpListener(IPAddress.Any, 80);  
listener.Start(5);
```

(2) Tiếp nhận yêu cầu kết nối TCP từ trình duyệt (hoặc bất kỳ chương trình nào khác)

```
var client = listener.AcceptTcpClient();
```

(3) Đọc dòng request line của truy vấn HTTP sử dụng adapter StreamReader

```
var sr = new StreamReader(client.GetStream());  
string request = sr.ReadLine();
```

(4) Tạo một chuỗi phản hồi đơn giản chứa status line và body là một chuỗi HTML, và gửi chuỗi phản hồi này trở lại cho client.

```
var sw = new StreamWriter(client.GetStream());  
sw.WriteLine("HTTP/1.1 200 OK\n");  
sw.WriteLine("<html><head></head><body><h1>Hello world</h1><body></html>");
```

Như vậy, bạn có thể thấy, chương trình do chúng ta viết ra có thể tương tác với trình duyệt thông qua giao thức HTTP. Bạn cũng có thể thấy, chuỗi HTML cũng được chứa bên trong phản hồi. Bên trong phản hồi này có thể chứa mã JavaScript, CSS, hoặc bất kỳ thứ gì có thể chuyển thành văn bản.

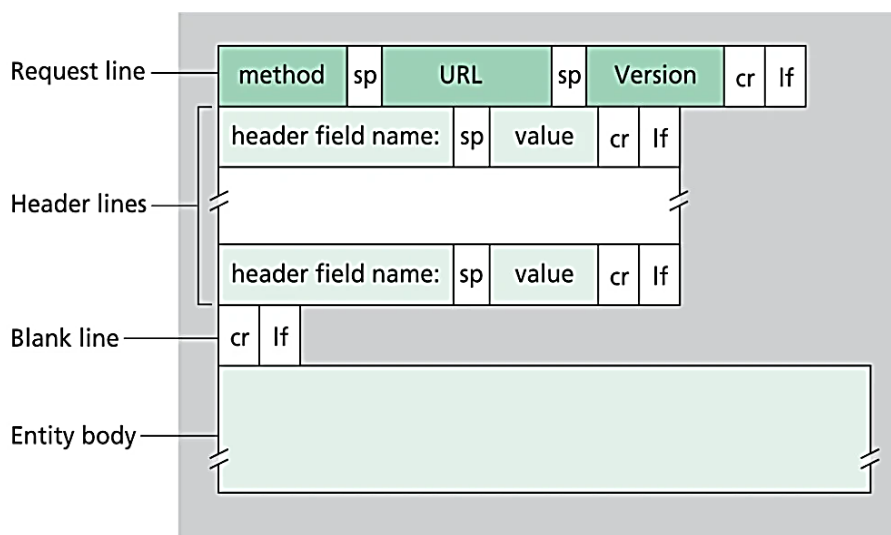
c. Cấu trúc gói tin của HTTP

Như ở trên đã trình bày, giao thức HTTP có hai loại gói tin với cấu trúc khác nhau: gói tin truy vấn (HTTP request) và gói tin phản hồi (HTTP response). Mỗi loại gói tin thực chất chỉ là các chuỗi văn bản có định dạng xác định.

Gói tin truy vấn của HTTP bao gồm các phần sau (hình 12.19):

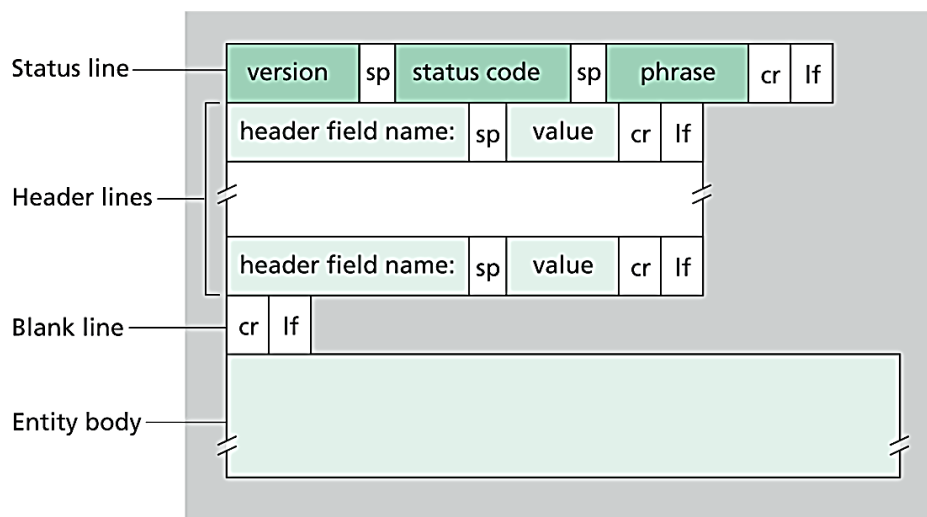
- Method (phương thức): xác định hành động mà client muốn thực hiện trên tài nguyên của server, ví dụ như GET (để lấy thông tin), POST (để tạo mới tài nguyên), PUT (để cập nhật tài nguyên), DELETE (để xóa tài nguyên), và nhiều phương thức khác.
- URL (Uniform Resource Locator): xác định địa chỉ của tài nguyên mà client muốn truy cập trên server.
- Headers (tiêu đề): chứa các thông tin về yêu cầu của client, ví dụ như kiểu dữ liệu yêu cầu, ngôn ngữ yêu cầu, thông tin xác thực, và các thông tin khác.

Body (nội dung): chứa các thông tin cụ thể mà client muốn gửi đến server, ví dụ như dữ liệu form, file tải lên, v.v.



Hình 0.3. Cấu trúc gói tin truy vấn HTTP

Gói tin phản hồi của HTTP bao gồm các phần sau (Hình 12.20):



Hình 0.4. Cấu trúc gói tin phản hồi HTTP

- 9) Status code (mã trạng thái): xác định kết quả của yêu cầu của client, ví dụ như thành công (200 OK), tài nguyên không tìm thấy (404 Not Found), lỗi server (500 Internal Server Error), v.v.
- 10) Headers (tiêu đề): chứa các thông tin về phản hồi của server, ví dụ như kiểu dữ liệu trả về, ngôn ngữ phản hồi, thông tin xác thực, và các thông tin khác.
- 11) Body (nội dung): chứa các thông tin cụ thể mà server muốn trả về cho client, ví dụ như nội dung HTML, JSON, XML, v.v.

Như vậy, nếu biết cấu trúc chuỗi văn bản này và biết cách lập trình với giao thức TCP, chúng ta hoàn toàn có thể tự xây dựng một chương trình web server đơn giản, hoặc có thể xây dựng một chương trình để phát truy vấn HTTP tới chương trình chủ.

Truyền dữ liệu qua HTTP

Giao thức HTTP được sử dụng để truyền tải nhiều loại dữ liệu khác nhau giữa client và chương trình chủ web. Các loại dữ liệu thông dụng nhất bao gồm HTML, CSS, JavaScript, file ảnh, dữ liệu dạng JSON, dữ liệu dạng XML.

Truyền các loại dữ liệu web truyền thống

HTML (HyperText Markup Language) là loại ngôn ngữ dùng để tạo ra nội dung trên web. Bản thân dữ liệu HTML là văn bản, chứa các đánh dấu (markup), do vậy, có thể dễ dàng nhúng vào phần body của gói tin HTTP response để trả cho trình duyệt.

CSS (Cascading Style Sheets) là ngôn ngữ dùng để mô tả cách trình bày và hiển thị các trang web được viết bằng HTML. File CSS thực chất cũng là một file văn bản, vì vậy nó có thể được nhúng trực tiếp vào phần body của gói tin HTTP response.

JavaScript là loại ngôn ngữ lập trình được sử dụng phổ biến trên trình duyệt web để tạo ra các ứng dụng web động và tương tác trên trang web. File JavaScript có phần mở rộng js cũng là một file văn bản. Phản hồi của HTTP chứa mã nguồn của JavaScript trực tiếp trong phần body để gửi cho trình duyệt.

Thông thường, một trang web được tạo ra từ một file HTML (chứa nội dung). Mỗi file HTML thường tham chiếu tới rất nhiều file CSS (chứa mô tả cách trình bày trang) và JavaScript (tạo tương tác động). Mỗi khi file HTML được tải về, trình duyệt sẽ căn cứ vào các tham chiếu chứa trong HTML để lần lượt tải tất cả các file có liên quan về. Chỉ khi nào tải được đầy đủ các file, trang web mới thực sự hoạt động.

Truyền các loại dữ liệu đặc biệt

Với các ứng dụng đơn trang, ứng dụng desktop, hoặc ứng dụng mobile, giao thức HTTP không được sử dụng để truyền tải các file. Do HTTP sử dụng văn bản cho cấu trúc gói tin, bất kỳ văn bản nào cũng có thể gửi qua giao thức này. Có hai định dạng văn bản đặc biệt được sử dụng phổ biến: JSON và XML. Chương trình server gửi các chuỗi ký tự JSON hoặc XML bên trong phần body.

JSON là một loại định dạng dữ liệu được viết theo cú pháp tạo đối tượng của ngôn ngữ JavaScript. Thực chất JSON cũng chỉ là một chuỗi ký tự được chương trình server sinh ra theo yêu cầu từ client.

XML là một loại ngôn ngữ định dạng dữ liệu khác cũng ở dạng văn bản. Chuỗi XML được chương trình server nhúng vào phần body của gói tin response.

Hai loại văn bản đặc biệt này được sử dụng để định dạng dữ liệu trao đổi giữa ứng dụng Web API và ứng dụng khai thác nó, như ứng dụng mobile, ứng dụng desktop, ứng dụng web đơn trang. Cách xây dựng ứng dụng dạng Web API sẽ được xem xét trong một bài học riêng.

Truyền dữ liệu nhị phân

Dữ liệu nhị phân là những loại dữ liệu khác mà không thể hiện trực tiếp thành văn bản. Dữ liệu nhị phân thường dùng để lưu trữ các tập tin ảnh, âm thanh, video, phần mềm, v.v..

Đối với **dữ liệu nhị phân**, cần phải có cách thức riêng để chuyển đổi nó thành văn bản trước khi có thể nhúng vào thân của truy vấn / phản hồi HTTP. Để chuyển đổi file nhị phân thành text để truyền qua giao thức HTTP, bạn có thể sử dụng thuật toán mã hóa nhị phân sang Base64.

Base64 là một thuật toán mã hóa dữ liệu nhị phân thành dạng văn bản có thể đọc được bằng cách sử dụng bảng ký tự định sẵn. Trong quá trình chuyển đổi, các ký tự không hợp lệ hoặc không thể đọc được sẽ được thay thế bằng các ký tự an toàn.

Để chuyển đổi file nhị phân sang Base64, bạn có thể sử dụng các thư viện mã hóa Base64 có sẵn trong các ngôn ngữ lập trình. Sau khi chuyển đổi file nhị phân sang chuỗi Base64, bạn có thể truyền dữ liệu qua giao thức HTTP như với bất kỳ dữ liệu văn bản nào.